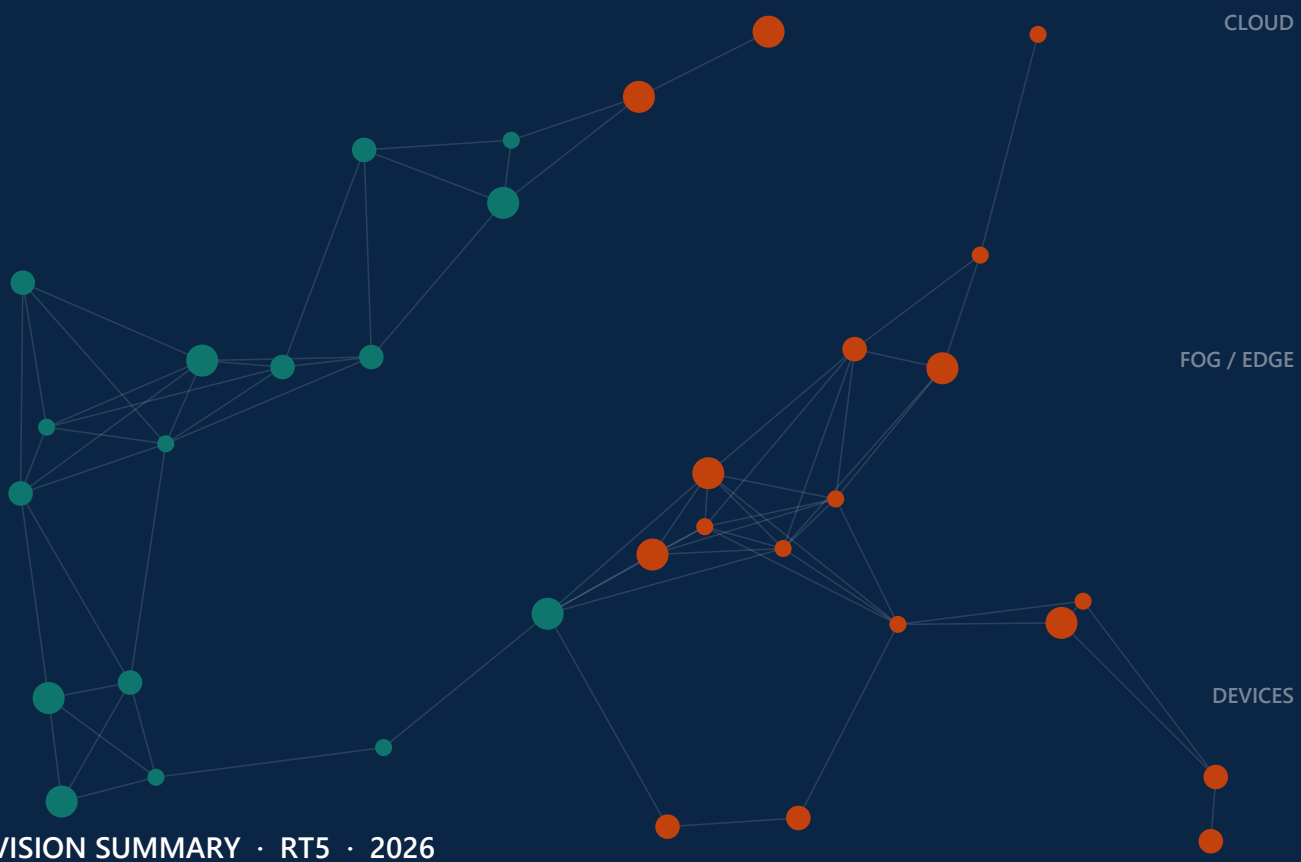


REVISION SUMMARY · RT5 · 2026

Technologies & Protocols for the Internet of Things

CHAPTER 1

Introduction to the IoT

Benefits and domains · architecture and layers · fog and M2M · World Forum models · devices · ZigBee, Wi-Fi, Bluetooth, LPWAN · REST, CoAP, MQTT

CHAPTER 2

Edge Computing for the IoT

Edge and Edge AI · compute continuum · ETSI MEC · Kubernetes, K3s, KubeEdge · intelligent edge · task offloading with deep RL · queueing models

- Diagrams redrawn from the course
- Comparison tables and formulas
- Exam tips, worked examples
- Review questions with answers
- Cheat sheet and glossary
- DS 2022 with a full correction

Based on the course by Dr. Sofiane Ouni · Technologies et Protocoles pour l'IoT (2026)

Contents

Chapter 1 · Introduction to the IoT 3
1.1 Why the IoT matters . . . 3
1.2 Application domains . . . 4
1.3 Case studies from the course . . . 6
1.4 IoT architecture and layers . . . 7
1.5 Communication models: the IoT World Forum reference . . . 8
1.6 IoT devices . . . 9
1.7 Network connections . . . 10
1.8 Application protocols: REST, CoAP and MQTT . . . 12
1.9 Chapter 1 recap and review questions . . . 13

Chapter 2 · Edge Computing for the IoT 14
2.1 Edge computing: what and why . . . 14
2.2 Edge AI and machine learning at the edge . . . 15
2.3 The compute continuum, data flow and edge products . . . 15
2.4 Multi-access Edge Computing (MEC) and ETSI . . . 16
2.5 Kubernetes at the edge . . . 18
2.6 Kubernetes fundamentals . . . 19
2.7 Kubernetes at the edge: challenges and benefits . . . 21
2.8 Lightweight Kubernetes distributions and managed edge services . . . 22
2.9 K3s in practice . . . 23
2.10 KubeEdge . . . 24
2.11 The intelligent edge . . . 26
2.12 Task offloading . . . 27
2.13 Reinforcement learning primer (appendix of the course) . . . 28
2.14 Deep Q-learning for vehicular task offloading . . . 30
2.15 Queueing models for edge and fog . . . 30
2.16 RL-based Kubernetes scheduling . . . 32
2.17 Chapter 2 recap and review questions . . . 32

Cheat sheet: the numbers and lists to know 34

Glossary of acronyms 35

DS 2022 — proposed correction 36
Exercise 1 · MQTT, CoAP, REST and MQTT-SN (10 points) . . . 36
Exercise 2 · Drones over MQTT, then REST (10 points) . . . 39

NOTE · HOW TO USE THIS SUMMARY

Chapters have their own colour: **teal = Chapter 1** (IoT), **orange = Chapter 2** (edge computing).

Boxes: **KEY IDEA** the essentials · **EXAM TIP** traps and comparisons · **REMEMBER** numbers to know · **EXAMPLE** worked cases · **NOTE** extra context beyond the slides.

Each chapter ends with a recap and review questions. Read the question, answer it, then check. The last pages hold a one-page cheat sheet, a glossary and a proposed correction of the DS 2022 exam.

CHAPTER 1

Introduction to the IoT

Why the Internet of Things matters, where it is used, how an IoT system is organised (layers, fog, communication models), what IoT devices look like, and how they talk to each other (networks and application protocols).

KEY IDEA · CHAPTER IN A NUTSHELL

The **Internet of Things (IoT)** connects physical objects (sensors, actuators, machines, vehicles, meters…) to networks so that they can report what they sense and be controlled remotely. Value comes from turning that data into **decisions**: locally (edge / fog), or centrally (cloud).

This chapter answers four questions: **why** (impact and market), **where** (application domains and case studies), **how it is built** (layers, fog, World Forum models, devices) and **how it communicates** (links such as ZigBee, Wi-Fi, LPWAN and application protocols such as MQTT and CoAP).

1.1 Why the IoT matters

IoT applications reach almost every sector: **healthcare**, **industrial automation** (Industry 4.0), **energy management and smart grids**, **transportation**, **smart infrastructure** (smart home, smart city), **retail** and more. The course quotes a global economic impact of **$4 to $11 trillion per year by 2025**.

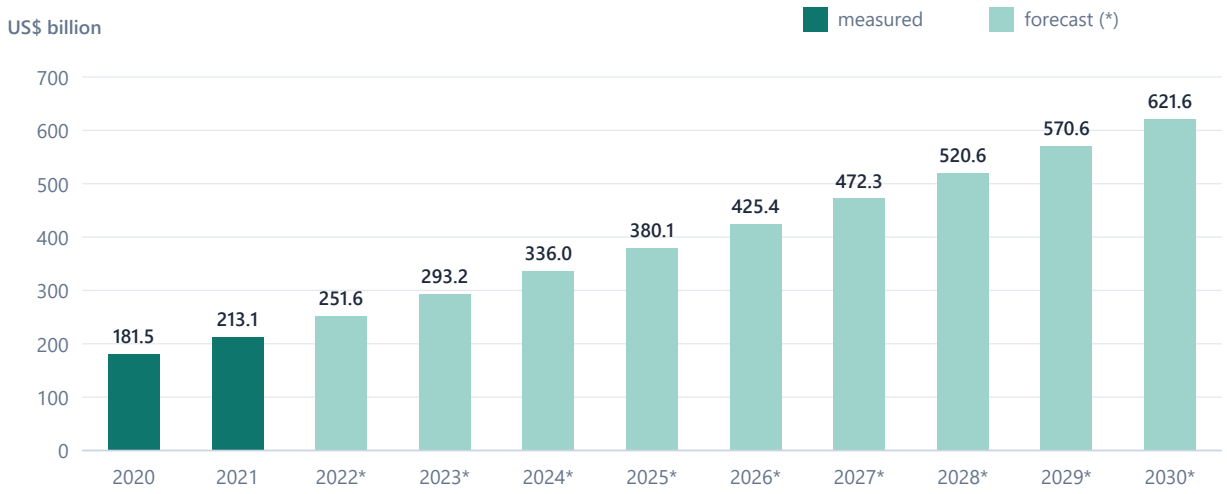


Figure 1.1 *— IoT market revenue worldwide, 2020–2030 (US$ billion; * = forecast; source: Transforma Insights / Statista 2023).*

Benefit	What it means in practice
Optimised resource usage	Sensors show exactly where water, energy or material is used, so resources are spent only where needed — sustainable growth for future generations.
Lower time and costs	Process optimisation (automatic monitoring, fewer manual checks, less waste) creates new savings.
Better decision making	Insights from **real-time data** support faster, better decisions and enhanced governance.
Innovative services	New functions and better user experiences are easy to create, improving quality of life.

Table 1.1 *— The four benefits of the IoT highlighted in the course.*

REMEMBER

Revenue grows from **$181.5 B (2020)** to a forecast **$621.6 B (2030)**: roughly ×3.4 in ten years, i.e. about +13 % per year.

Economic impact of IoT applications: **$4–11 trillion per year by 2025**.

1.2 Application domains

The course groups IoT applications into eight families. Each one has a few typical uses worth remembering:

Domain	Typical applications
Transport & logistics	Fleet management, goods tracking
Utilities	Smart metering, smart grid management
Smart cities	Parking sensors, waste management
Smart building	Smoke detectors, home automation
Consumers	Wearables, kids / senior trackers
Industrial	Process monitoring and control, maintenance monitoring
Environment	Food monitoring and alerts, environmental monitoring
Agriculture	Climate and agricultural monitoring, livestock tracking

Table 1.2 *— IoT application domains and examples.*

Popularity (what people searched and talked about) is not the same as real deployments. In Q3/2016, a count of **640 real IoT projects** gave a different ranking, with clear regional differences:

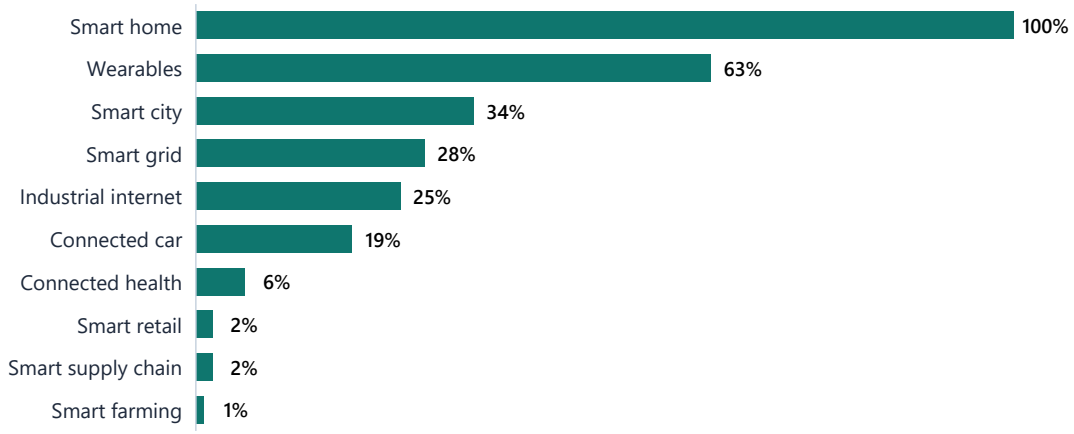


Figure 1.2 *— Popularity of IoT applications in Q4/2014 (relative popularity, smart home = 100 %).*

Segment	Share of projects	Americas	Europe	APAC
Connected industry	22 %	43 %	30 %	20 %
Smart city	20 %	31 %	**47 %**	15 %
Smart energy	13 %	49 %	24 %	25 %
Connected car	13 %	43 %	33 %	17 %
Other	8 %	46 %	33 %	13 %
Smart agriculture	6 %	48 %	31 %	17 %
Connected building	5 %	48 %	33 %	12 %
Connected health	5 %	**61 %**	30 %	6 %
Smart retail	4 %	52 %	30 %	13 %
Smart supply chain	4 %	57 %	35 %	4 %

Table 1.3 *— Top IoT segments by number of projects, Q3/2016 (N = 640), with the regional split.*

EXAM TIP

Know the contrast: **smart home** was the most *popular* topic (2014), but **connected industry (22 %)** and **smart city (20 %)** had the most *projects* (2016). The Americas lead most segments; **Europe leads smart city** projects (47 %).

Smart cities: the richest application area

A smart city combines many sub-systems, each fed by its own sensors. The course lists seven areas:

Area	What is sensed or controlled
Smart homes	Demand response, fire detection, temperature monitoring, security systems, social-network support
Smart parking lot	Number of cars, departures and arrivals, environment monitoring, mobile ticketing, traffic-congestion control
Healthcare	Tracking, identification, data gathering, sensing
Weather & water	Weather conditions, water quality, leakage, water level, contamination
Transportation & traffic	Camera monitoring, environment monitoring, travel scheduling, traffic-jam reduction, assisted driving
Environmental pollution	Greenhouse-gas monitoring, energy efficiency, renewable energy use, air quality, noise pollution
Surveillance	CCTV, violence detection, public-place monitoring, people and object tracking, traffic police

Table 1.4 *— Smart-city application areas.*

- Structural health of buildings
- Waste management
- Air quality and noise monitoring
- Traffic congestion
- City energy consumption
- Smart parking and smart lighting
- Automation of public buildings

The smart pole concentrates many city services on one street pole: sensors, video monitoring, RFID / manhole monitoring, emergency call, intelligent lighting, Wi-Fi / cellular access, information display and an EV charging pile.

Expected benefits: lower energy use, lower CO_2 emissions, better citizen safety and a higher quality of life.

1.3 Case studies from the course

Retail supply chain: RFID-tracked fish in Vietnam (IBM)

IBM's RFID-based IoT solution tracks fish at **every point of the supply chain**. Each fish carries a tag that records its **origin**, **catch date** and the **temperature during transit**, so that freshness can be checked at the checkout. Food spoilage costs about **$2 trillion per year** worldwide, while an RFID tag costs **less than 10 cents**: a very cheap way to make food safer.

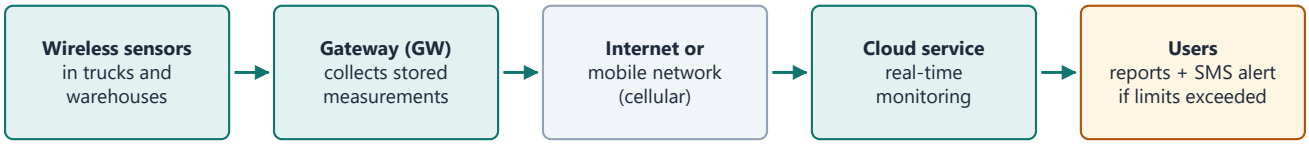
RFID for logistics

In a warehouse (e.g. IRG System, REAP solutions) goods are read either with a **mobile RFID reader** or automatically when they pass an **RFID gate**: pallets, boxes and individual items are identified in real time, without line of sight, which removes manual scanning and stock errors.

Cold chain for pharmaceuticals: Seemoto

Seemoto is a **real-time temperature-monitoring** solution for the pharmaceutical supply chain. Robust wireless sensors send their measurements to cloud services **through gateway devices** using the Internet or mobile-phone networks.

The cloud service produces reports (measurements, routes, deliveries, alerts) and sends **SMS alerts** to the driver and the manager as soon as a limit is exceeded, so that the cold chain can be qualified and corrected in time.



Seemoto: measured temperature data travel from sensor to cloud; the driver and the manager are alerted by SMS when a threshold is crossed

Figure 1.3 *— Seemoto data path: sensor → gateway → network → cloud → users.*

Four classic edge-computing examples (Digi)

Example	What the IoT does
Self-driving cars	LiDAR scanners, sensors and connected-vehicle technology; data processed in real time by onboard computers to decide navigation, speed and driving behaviour (better routes, fuel efficiency, road safety).
Smart grids	Smart meters measure energy use at fixed intervals and transmit it wirelessly: utilities optimise distribution, balance loads, detect faults.
Predictive maintenance	Sensors monitor equipment health in real time (temperature, vibration, energy consumption) to repair before a failure.
Agriculture	Autonomous recording stations give real-time soil, weather and crop-health data; farmers plan irrigation, fertilisation and pest control remotely.

Table 1.5 *— Examples of IoT with processing at the edge.*

1.4 IoT architecture and layers

An IoT system is usually described as a stack of four layers. Data flows upward (from sensors to the cloud) and commands flow downward (from applications to actuators).

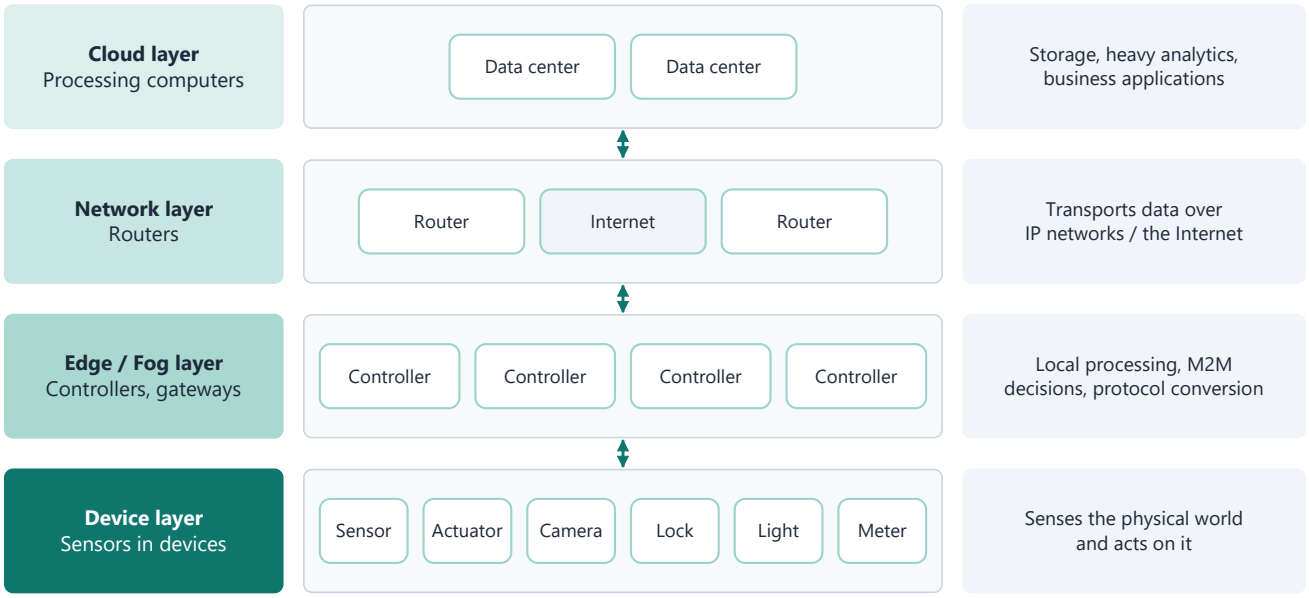


Figure 1.4 *— The four IoT layers and the role of each one.*

Layer	Equipment	Role
Device	Sensors and actuators embedded in things	Measure the physical world and act on it
Edge / fog	Controllers, gateways	Collect data, process it locally, take quick decisions, convert protocols, filter what goes up
Network	Routers, switches, the Internet	Transport data between sites and the cloud
Cloud	Data centers, processing computers	Long-term storage, heavy analytics, business applications, dashboards

Table 1.6 *— Summary of the IoT layers.*

Fog computing and machine-to-machine (M2M) communication

The course illustrates fog computing with **smart traffic lights**. Sensors detect and report traffic activity to a **controller**; the controller processes the data **locally**, computes the best traffic pattern and sends signals to the **actuators** in the lights to adjust traffic flows.

This is **machine-to-machine (M2M)** communication: sensors, actuators and controller all live in the fog, so the information is **not sent beyond the local network** of end devices. The same pattern appears in agriculture: an **IP-enabled controller** (e.g. Arduino or Raspberry Pi) reads soil sensors and drives irrigation actuators over IEEE 802.15 links, and reaches the Internet through Wi-Fi, cellular or a wired connection.

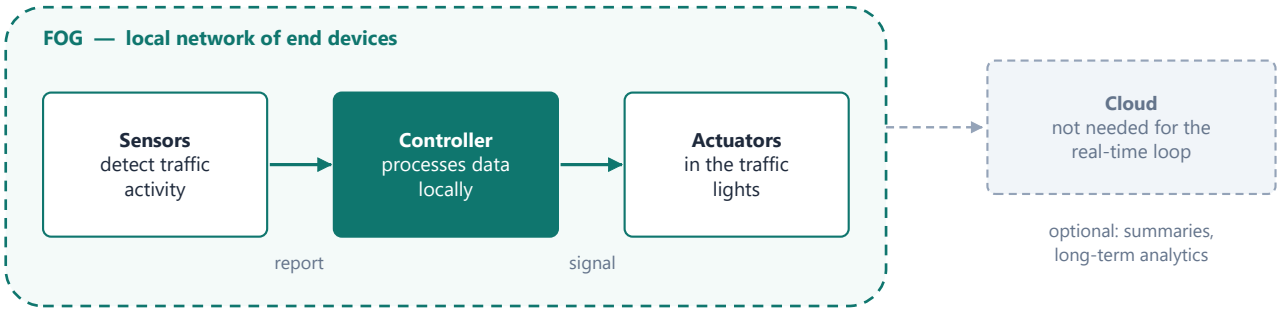


Figure 1.5 *— Smart traffic lights: an M2M loop that stays inside the fog.*

KEY IDEA

Why process in the fog rather than in the cloud?

- **Latency**: a local loop reacts in milliseconds; a cloud round trip can take hundreds.
- **Bandwidth**: only summaries or alarms leave the site.
- **Autonomy**: the loop keeps working if the Internet link is down.
- **Privacy**: raw data can stay on the premises.

1.5 Communication models: the IoT World Forum reference

The simplified IoT architecture of the **IoT World Forum reference model** describes four ways of connecting a device to the services that use its data. From model 1 to model 4, each step adds one more element between the device and the user.

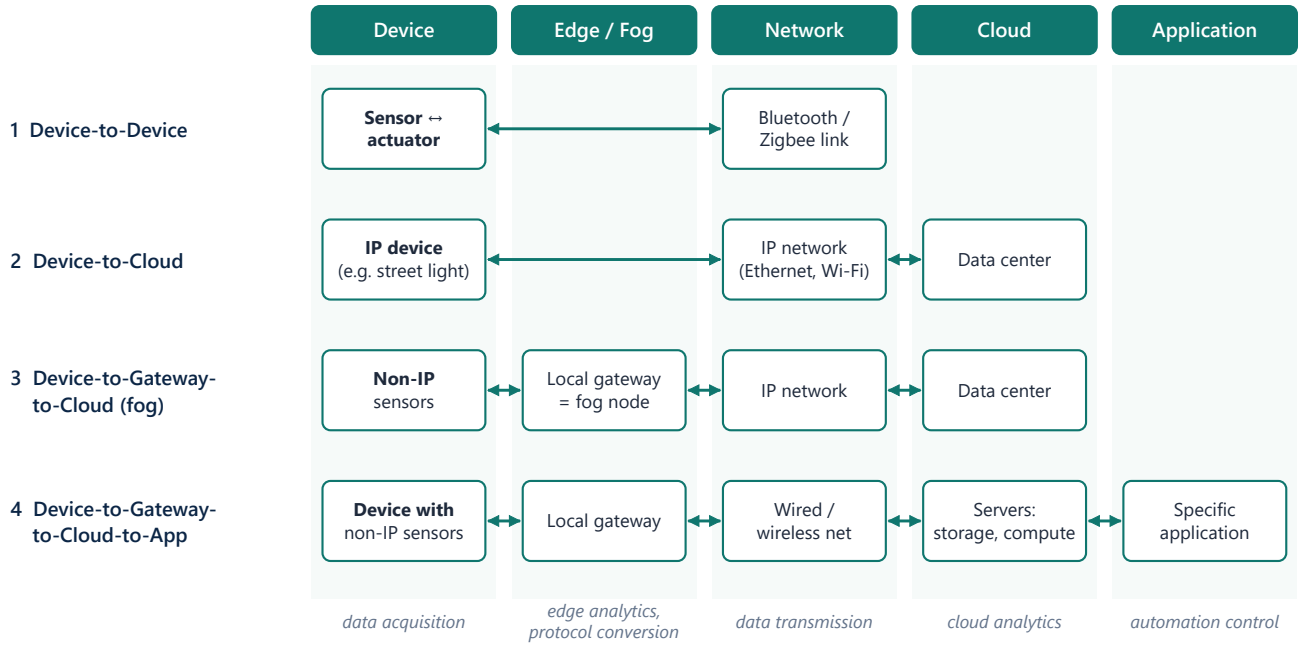


Figure 1.6 *— The four communication models of the IoT World Forum reference model and the layers they use.*

Model	How it works	Typical example
1 · Device → device	Two smart objects talk **directly** over a wireless protocol such as Bluetooth or ZigBee.	A dry-earth sensor triggers a watering system.
2 · Device → cloud	The device connects through a local network **directly to an Internet cloud service** (wired Ethernet or Wi-Fi). Device, IP network and cloud exchange data and control messages.	Traffic-volume sensors or street lights reporting to a data center.
3 · Device → gateway → cloud (fog)	Non-IP devices reach the cloud through an **application on a local gateway**, which can also provide security and data / protocol translation (edge analytics).	A smartphone app acting as gateway for a fitness tracker.
4 · Device → gateway → cloud → application	Data is collected through a gateway to a local IP network, flows to the fog or cloud, and is **exported and analysed by users**, often combined with data from other sources.	Automation control applications.

Table 1.7 *— IoT World Forum communication models.*

EXAM TIP

Models 3 and 4 both use a gateway. The difference: model 4 adds the **application layer**, where the data is exported, combined with other sources and analysed by users.

1.6 IoT devices

IoT devices are mostly **embedded systems** with small amounts of memory and limited power. Their constraints shape every protocol choice made later in the chapter:

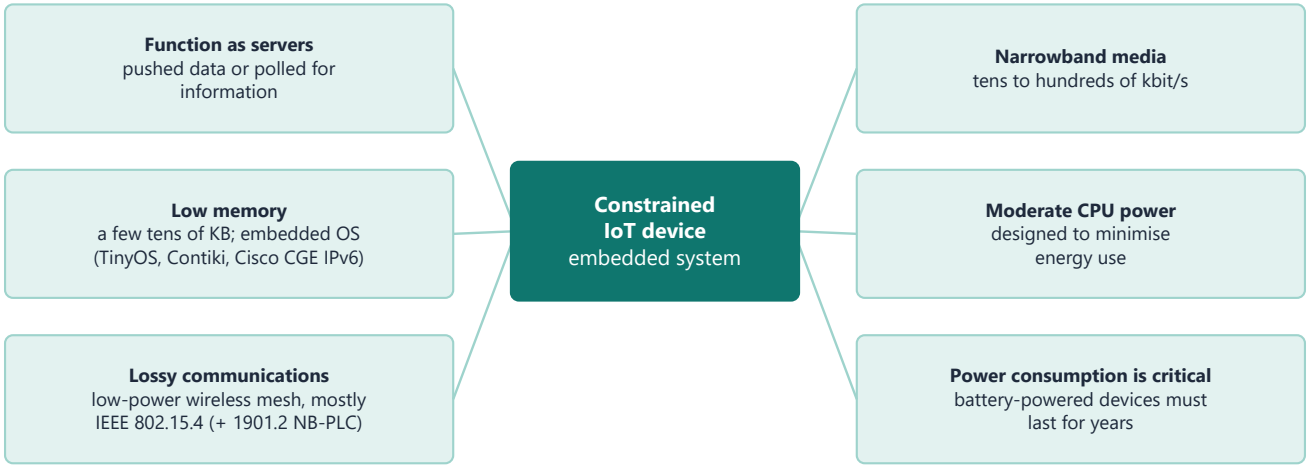


***Figure 1.7** — The six typical constraints of an IoT device.*

Constraint	Detail from the course
Function as servers	Devices may push data or be polled for information, so they often run a small server.
Low memory	A few tens of KB of memory, with small embedded operating systems such as TinyOS, Contiki or the Cisco CGE IPv6 SDK.
Lossy communications	Low-power wireless mesh links, mostly IEEE 802.15.4 (802.15.4g / 802.15.4e-2012), or narrowband power-line communication IEEE 1901.2 (NB-PLC).
Narrowband media	Data rates of tens to hundreds of kbit/s.
Moderate CPU	Processors are chosen to minimise energy use, not to maximise speed.
Power consumption is critical	Battery-powered devices must last for **years** without maintenance.

***Table 1.8** — Constraints of IoT devices.*

Popular hardware platforms named in the course fall into three groups:

- **Microcontroller boards** (tiny, low power, no full OS): Arduino, ESP8266 (with built-in Wi-Fi) and its NodeMCU development kit, Flutter (long-range wireless board).
- **Single-board computers** (run Linux, more power): Raspberry Pi, BeagleBone, Banana Pi, CHIP, Intel Edison and Intel Galileo.
- **Sensor-network motes** (made for low-power wireless sensor networks): WaspMote, openMote.

1.7 Network connections

Devices can only exchange messages if they share the same **protocols**, the agreed rules of communication. At the data-link level, IoT relies mainly on local wireless standards: **IEEE 802.11 (Wi-Fi)**, **IEEE 802.15 (Bluetooth)** and **IEEE 802.15.4 (ZigBee)**. Newer technologies cover WAN and MAN distances:

- **LoRaWAN**: a low-power wide-area network (LPWAN).
- **Narrowband IoT (NB-IoT)**: a cellular way to reach things that send small amounts of data, over long periods, in hard-to-reach places.

Constrained vs unconstrained communication

	Unconstrained networks	Constrained networks
Examples	LAN, MAN, WAN: Ethernet, Wi-Fi, fibre, broadband over power lines (BPL)	IEEE 802.15.4, Bluetooth, low-power 802.11, RFID
Reliability / latency	High reliability, low latency	Lossy links, long latency
Data rate	High data rates	Low: about **1 Mbit/s or less**
Energy	Complex nodes, high energy use	Low energy use, power-saving modes, sleeping nodes

Table 1.9 *— Unconstrained vs constrained communication.*

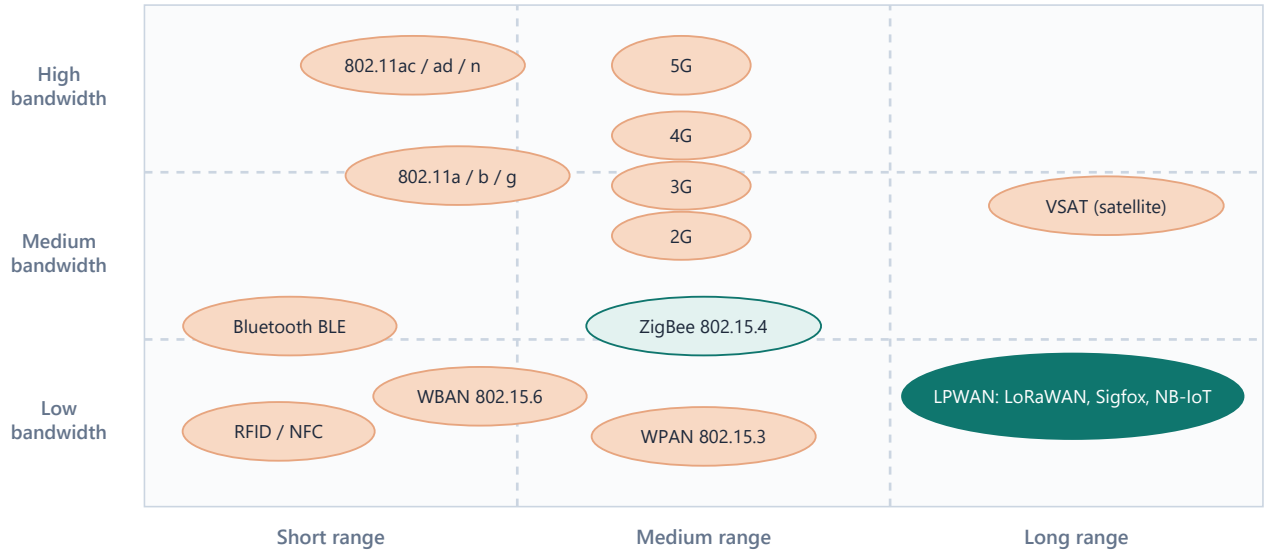


Figure 1.8 *— Wireless technologies positioned by range and bandwidth (simplified view of the course figure).*

	ZigBee (802.15.4)	Wi-Fi (802.11)	Bluetooth (802.15.1)
Main use	Sensor networks, low power	High-speed Internet access	Device-to-device connectivity
Battery life	**Years**	Hours	About 1 week
Data rate	250 kbit/s	Up to 54 Mbit/s	720 kbit/s
Range	Up to 100 m	100 m and more	10–100 m
Strength	Low power, low cost	Speed, ubiquity	Convenience

Table 1.10 *— Comparison of the three local wireless standards.*

Low-Power Wide-Area Networks (LPWAN)

An LPWAN is designed for **long-range communication of low-bit-rate devices** (sensors, actuators, controllers). Coverage is excellent in urban, rural and remote areas: the signal reaches **5 to 40 km** depending on how open the terrain is. Base stations have a **battery life of about 10 years** and support **thousands of devices**. Examples: **LoRaWAN, Sigfox, Weightless-N, RPMA**.

> **REMEMBER**
> Short range, low power: **Bluetooth / BLE, ZigBee**. High throughput: **Wi-Fi**. Long range, very low throughput: **LPWAN** (LoRaWAN, Sigfox, NB-IoT…), 5–40 km, years of battery life.

1.8 Application protocols: REST, CoAP and MQTT

The IoT supports three kinds of application connections: **machine to machine (M2M)**, **machine to gateway (M2G)** and **machine to cloud (M2C)**. In every case both ends must speak the same application-layer protocol. Three protocols dominate:

Protocol	Definition (course)
REST (RESTful web services)	A style of API that makes it easy for programs to interact over the Internet, using **HTTP and URLs** to request web services.
CoAP — Constrained Application Protocol	A **RESTful web transfer protocol for constrained nodes and networks**. Like HTTP, resources are identified by URIs and handled with GET, PUT, POST and DELETE. Supports **multicast**, low overhead, simplicity, and can even run over SMS. Works on micro-controllers with **10 KB of RAM and 100 KB of storage**, with strong security.
MQTT — Message Queuing Telemetry Transport	A **lightweight publish / subscribe** messaging protocol with minimal overhead, high data integrity and security for remote environments; best for low-bandwidth links.

Table 1.11 *— The three main IoT application protocols.*

How MQTT works. An intermediate node, the **message broker**, sits between producers and consumers. Every data source **publishes** each data element to the broker under a **topic**; nodes interested in a topic **subscribe** to it; the broker then distributes every message to the clients subscribed to its topic. Publishers and subscribers never need to know each other.

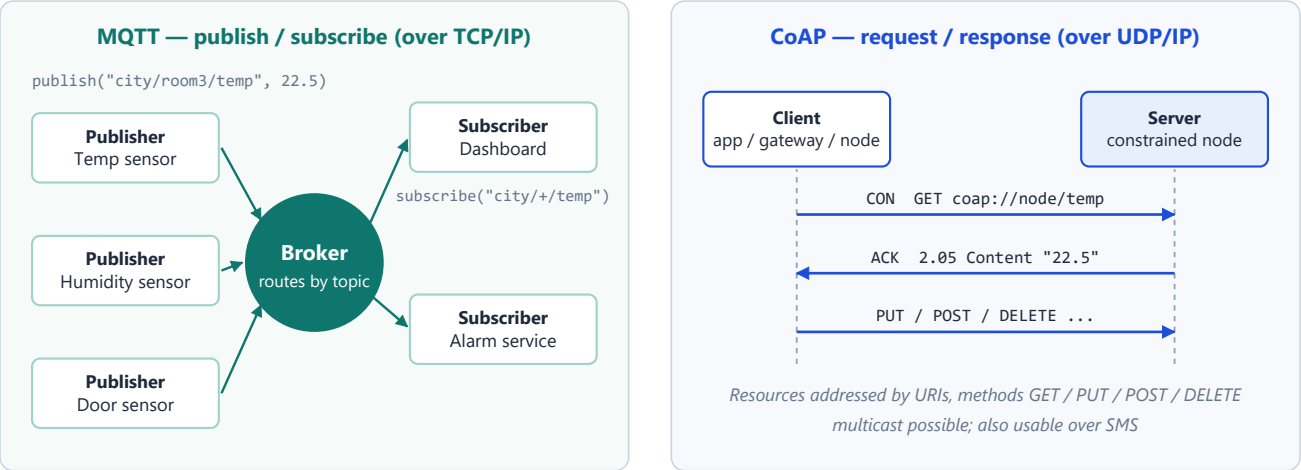


Figure 1.9 *— MQTT (publish / subscribe through a broker) vs CoAP (request / response between client and server).*

	MQTT	CoAP
Pattern	Publish / subscribe via a broker	Request / response (client–server), REST style
Transport	**TCP/IP** — connection-oriented	**UDP/IP** — connectionless
Advantages of the transport	Delivery is ensured; flow control	Lightweight, easy to implement
Drawbacks of the transport	More protocol data and processing; harder on some network topologies	Delivery not ensured; no sequencing of packets
Best for	Telemetry from many devices to many consumers over low-bandwidth links	Direct access to resources on very constrained nodes, multicast

Table 1.12 *— MQTT vs CoAP.*

NOTE · BEYOND THE SLIDES

Useful facts that are standard but **not on the slides**: MQTT uses TCP port 1883 (8883 with TLS) and offers three delivery levels, QoS 0 / 1 / 2; CoAP uses UDP port 5683 and makes UDP reliable when needed with *confirmable* (CON) messages answered by an ACK.

1.9 Chapter 1 recap and review questions

KEY IDEA · KEY POINTS

- IoT = sensors / actuators + connectivity + data processing; impact $4–11 T / year (2025); market $181.5 B (2020) → $621.6 B (2030, forecast).
- Eight domains; in 2016 the most projects were in connected industry (22 %) and smart city (20 %).
- Four layers: device → edge / fog → network → cloud. Fog = local processing, M2M loops.
- IoT World Forum models: D2D, D2C, D2G2C (fog gateway), D2G2C2App.
- Devices are constrained: tens of KB of RAM, lossy narrowband links, years on a battery.
- Links: ZigBee (250 kbit/s, years), Wi-Fi (54 Mbit/s, hours), Bluetooth (720 kbit/s, a week), LPWAN (5–40 km).
- Protocols: REST (HTTP), CoAP (REST over UDP for constrained nodes), MQTT (pub/sub over TCP with a broker).

Q1 Give the four benefits of the IoT presented in the course.

Optimised resource usage (sustainable growth), lower time and costs (process optimisation), better decision making from real-time data, and innovative services that improve quality of life.

Q2 What is the difference between the 2014 popularity ranking and the 2016 project ranking?

Popularity (2014) was led by the smart home (100 %) and wearables (63 %). Real projects (2016, N = 640) were led by connected industry (22 %) and smart city (20 %).

Q3 In the smart traffic-light example, why is the communication called M2M and fog-based?

Sensors, controller and actuators exchange data among themselves without a human; all three are in the local network (the fog), so the data never has to leave it.

Q4 Which World Forum model fits a fitness tracker connected through a phone?

Model 3, device-to-gateway-to-cloud: the smartphone application is the local gateway that relays (and can translate and secure) the tracker's data towards the cloud.

Q5 List the six constraints of IoT devices.

They act as servers, have low memory (tens of KB), use lossy communications (802.15.4, NB-PLC), narrowband media (tens to hundreds of kbit/s), moderate CPU, and power consumption is critical (years on battery).

Q6 Which technology would you choose for a soil sensor 10 km away that sends one value per hour?

An LPWAN such as LoRaWAN, Sigfox or NB-IoT: 5–40 km range, very low data rate, years of battery life. Wi-Fi and Bluetooth do not reach that far.

Q7 Explain the role of the broker and of topics in MQTT.

Publishers send each message to the broker with a topic; subscribers register interest in topics; the broker forwards each message to all subscribers of its topic. Producers and consumers are decoupled.

Q8 Why can CoAP run on smaller devices than HTTP, and what is its main transport drawback?

CoAP is a compact REST protocol over UDP (no TCP connection state), designed for 10 KB of RAM and 100 KB of storage. UDP does not guarantee delivery or order, so reliability must be added by the protocol when needed.

CHAPTER 2

Edge Computing for the IoT

Why processing moves to the edge, ETSI MEC, Kubernetes at the edge (K3s, KubeEdge), the intelligent edge, and task offloading with reinforcement learning and queueing models.



KEY IDEA · CHAPTER IN A NUTSHELL

IoT devices must react in **real time**, and there is no time for a round trip to a distant cloud. **Edge computing** moves processing close to the devices, at the edge of the network. The rest of the chapter answers: *where* is the edge (continuum, MEC), *how* do we run software there (containers, Kubernetes, K3s, KubeEdge), *what* intelligence runs there (Edge AI), and *how to decide* where each task should run (offloading, RL, queueing).

2.1 Edge computing: what and why

Edge computing processes data **near where it is produced** instead of in a centralised data center. It enables real-time experiences that were not possible before, and lets technical teams run a **self-managed Virtual Private Cloud (VPC) on relatively inexpensive hardware**. Cloud-native technologies such as **Kubernetes** make this edge infrastructure possible. The two advantages stressed in the course are **data privacy** and **bandwidth scalability**.

Use cases

- **IoT**: sensors and cameras process data locally for real-time responses.
- **Healthcare**: medical devices analyse patient data on site.
- **Autonomous vehicles**: real-time sensor processing for navigation and safety.
- **Industry 4.0**: real-time monitoring and optimisation of production.
- **Smart cities**: traffic lights and energy systems react to changing conditions.

Enabling technologies

- **IoT** devices generate data at the edge.
- **5G**: high bandwidth, low latency.
- **Virtualisation & containers** (Docker, Kubernetes) for deployment and scaling.
- **AI / ML** to analyse data locally.
- **Fog computing**: intermediate layer with extra processing and storage.
- **Advanced security** for devices and data.

A typical round trip from a connected device to a central cloud and back can reach **250 ms** (IBM). 5G alone improves this by only **about 2 %**; the big gain comes from moving workloads to the edge, where latencies **under 20 ms** are possible. CenturyLink targets **5 ms** for its edge, and Verizon projected **30 ms** when launching 5G. The intelligent industrial edge market (hardware, software, services) was **$11.6 B in 2019** and is expected to reach **$30.8 B by 2025**.

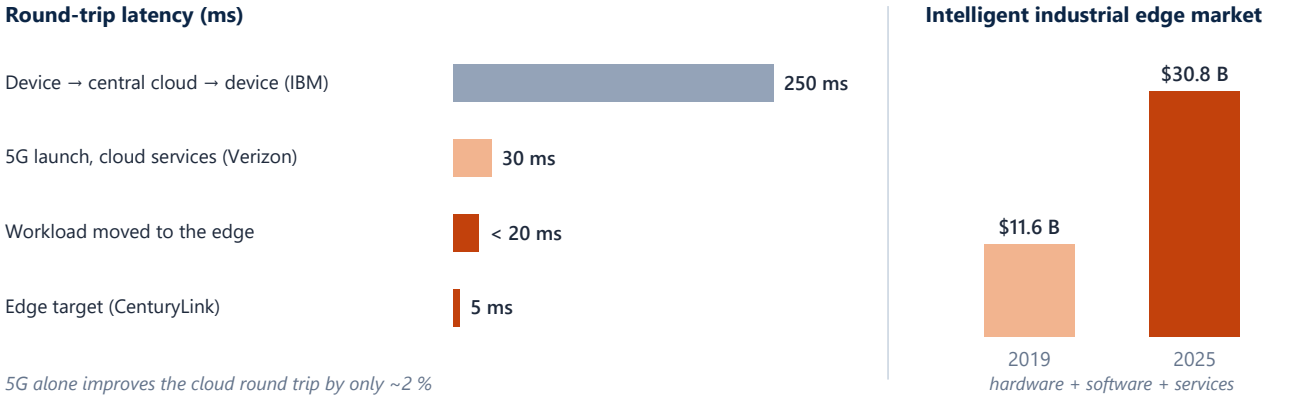


Figure 2.1 *— Why the edge: latency figures quoted in the course (left) and the intelligent industrial edge market (right).*

2.2 Edge AI and machine learning at the edge

AI brings advanced processing **to the source of the data**. The course lists five ways AI enhances edge computing, and four benefits of ML at the edge (which are the same ideas seen from the ML side):

Edge AI enhancement	Explanation	ML benefit
Reduced latency, real-time processing	Immediate analysis and decisions (autonomous vehicles, industrial automation).	Reduced latency
Improved privacy and security	Sensitive data (e.g. patient data) is not sent over the network.	Improved privacy
Bandwidth optimisation, cost reduction	Only relevant insights or aggregates go to the cloud.	Bandwidth savings
Reliability, offline functionality	Keeps working during network outages (remote or unstable sites).	Enhanced reliability
Scalability and flexibility	Models deployed and managed across many edge devices.	—

Table 2.1 *— Edge AI enhancements and the matching ML benefits.*

2.3 The compute continuum, data flow and edge products

"Edge" is not one place: between the sensor and the national data center there is a **continuum** of compute tiers. The closer to the data, the lower the latency but the smaller the resources.

A typical edge data flow keeps the fast loop on site. Machines send telemetry to the edge, which processes it, stores it, runs machine-learning models and sends **responses** straight back. Only **batched telemetry** is forwarded to a telemetry-ingestion endpoint in the cloud for long-term storage.

Category	Typical compute resources	Distance to data	Latency
Cloud	National data centers · regional data centers (core) · local data centers (aggregation)	10+ km	10+ ms
Thick edge	Cell-tower data centers (access) · on-premise data centers	1 – 10 km	1 – 10 ms
Thin edge	Computers · networking equipment · controllers	0.001 – 1 km	0.1 – 1 ms
Micro edge	Sensors / devices themselves	< 1 m	< 0.1 ms

Fog = the intermediate tiers between the cloud and the devices (extra processing and storage close to the data). Source: IoT Analytics 2020.

***Figure 2.2** — From the cloud to the micro edge: compute resources, distance to the data and latency (IoT Analytics, 2020).*

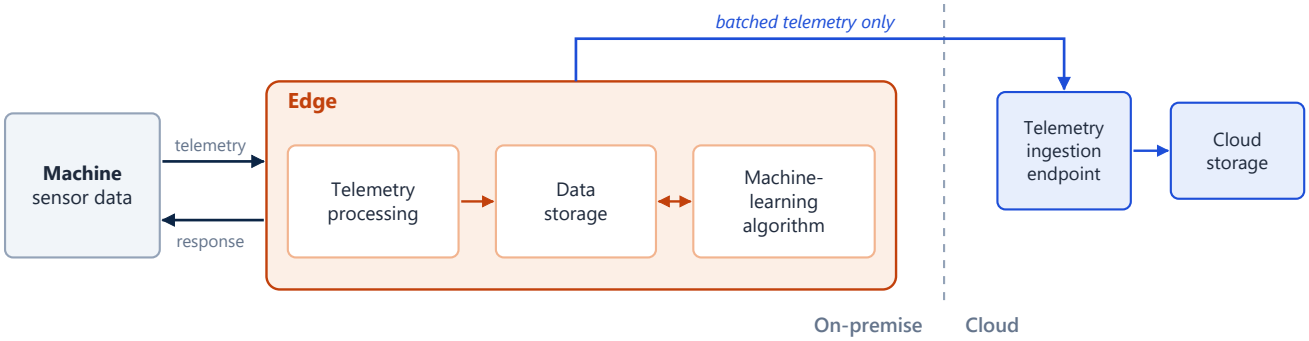


***Figure 2.3** — Edge data flow: the control loop stays on premise.*

Product (examples in the course)	What it is
Advantech UNO-238 V2	Compact industrial edge computer: Intel 12th-gen CPU (up to 10 cores), up to 64 GB DDR5, CAN bus.
Advantech UNO-220-P4N2	Raspberry Pi 4 gateway kit with PoE and a TPM security chip.
Welotec EG800	Industrial edge gateway certified for Azure IoT Edge.
WR302G-EC / WR312G-EC / WR322GR-EC	LTE / 5G edge platforms for embedded data acquisition, with Node-RED and MQTT.

***Table 2.2** — Edge hardware products cited in the course.*

2.4 Multi-access Edge Computing (MEC) and ETSI

MEC is the **ETSI** (European Telecommunications Standards Institute) standard for edge computing inside the operator network. It offers application developers and content providers **cloud-computing capabilities and an IT service environment at the edge of the network**, typically next to the radio access network.

- **Open standard**: several implementations, guaranteed interoperability.
- Built on the **ETSI NFV** framework (Network Functions Virtualisation), so MEC can be deployed in NFV environments.
- **Aligned with 3GPP**, which makes MEC a building block of **5G**.

- Key enabler for verticals: automotive, industrial automation, AR/VR, video streaming, gaming, e-health, smart cities.

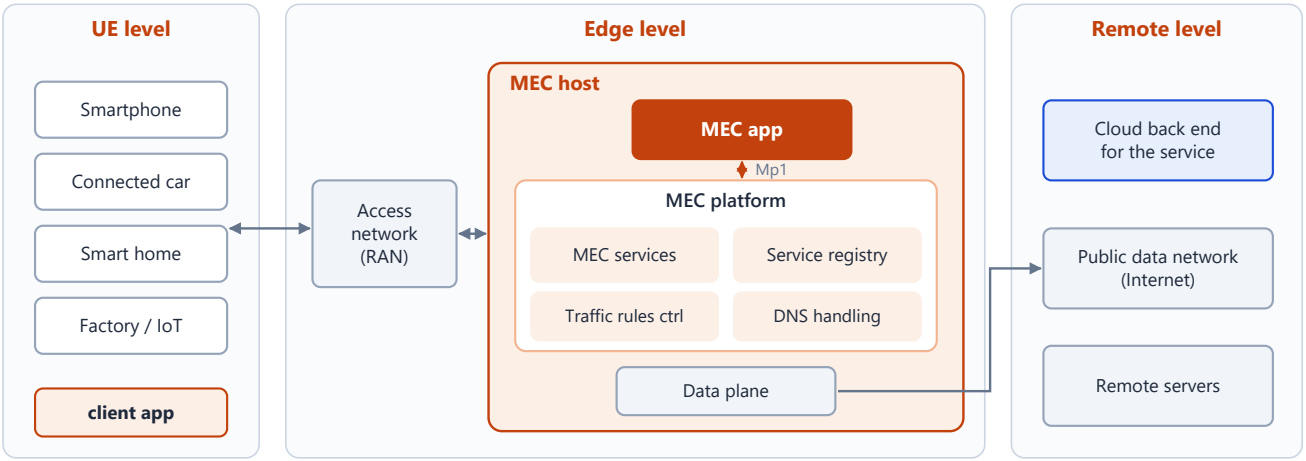


Figure 2.4 *— The three MEC levels: user equipment (UE), edge (MEC host) and remote.*

What an edge must offer: three dimensions

Dimension	Content
1 · Functional requirements	Latency-sensitive applications (AR/VR, CDN — content delivery networks); compute-intensive workloads (AR/VR); integration with **network APIs** (location, QoS, mobility management).
2 · Control and management	**Edge Applications as a Service** and **Edge Platform as a Service**.
3 · Network location and placement	On customer premises (enterprise vCPE — virtualised routers, firewalls —, set-top box); in the access network (RAN); in the core network; in a distributed cloud.

Table 2.3 *— MEC edge needs: a three-dimensional view.*

ETSI MEC reference architecture (GS MEC 003 V2.1.1, 2019-01)

MEC applications are **software-only entities** running on a virtualisation infrastructure located in or close to the network edge. Management is split into two levels:

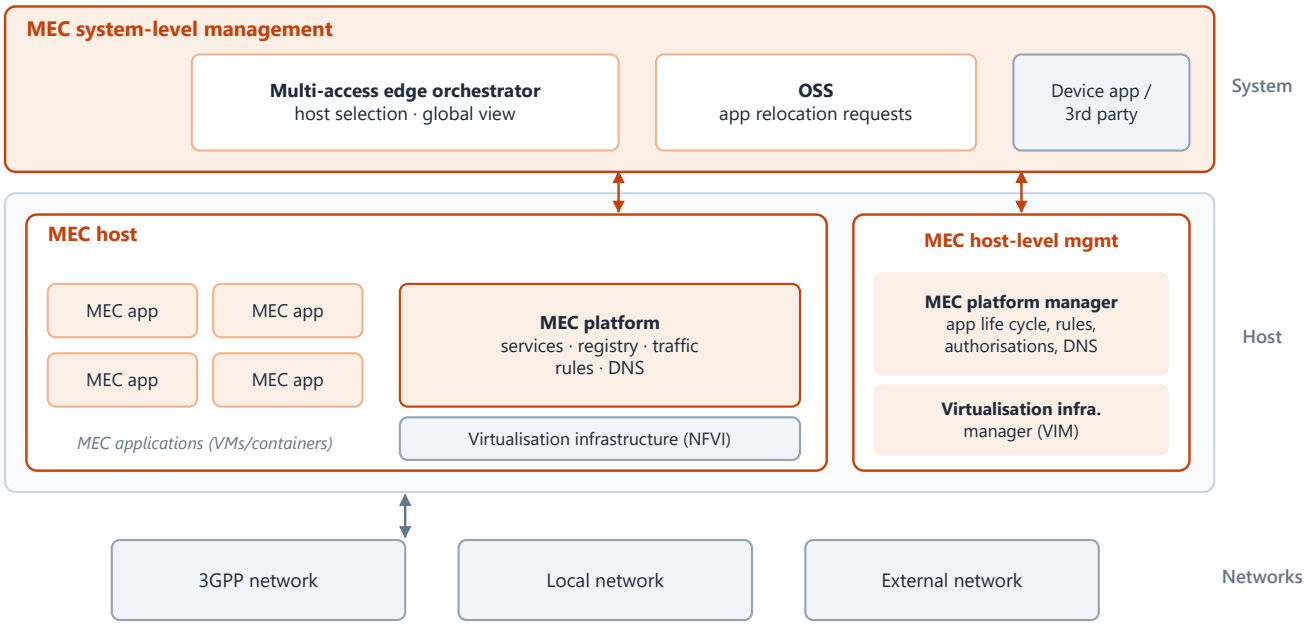


Figure 2.5 *— Simplified ETSI MEC reference architecture.*

Level	Entity	Responsibility
System	Multi-access edge orchestrator	Selects the MEC host(s) for each application according to latency, available resources and services; keeps an overall view of hosts, resources, services and topology.
System	Operations Support System (OSS)	Receives requests from device applications to relocate applications between external clouds and the MEC system.
Host	MEC platform manager	Application life cycle; application rules and requirements: service authorisations, traffic rules, DNS configuration, conflict resolution.
Host	Virtualisation infrastructure manager (VIM)	Manages virtualised compute, storage and network resources (ETSI NFV).
Host	MEC host = MEC platform + MEC apps + NFVI	The platform offers MEC services, a service registry, traffic-rule control and DNS handling to the apps.

Table 2.4 *— Main entities of the ETSI MEC architecture.*

Federation, cloud providers and edge-native applications

MEC federation connects the MEC systems of several operators through a **MEC federator**; a federator can also take the **federation broker** role and act as the hub of a hub-and-spoke topology. With **cloud service providers**, public clouds (AWS, Azure, Google Cloud) and operators both offer services at the **cloud edge** and the **cloud core**; a platform manager and cloud orchestration allocate resources according to application providers' requests.

Edge-native (cloud-native edge) applications use **serverless microservices**, **event-driven publish/subscribe** patterns, **synchronised databases** and **containers**: faster to deploy, better performance and resiliency. A flagship use case is **V2X cooperative perception**: vehicles detect the surrounding traffic in real time, exchange sensed data and share intended manoeuvres with minimum latency (European Commission goal for safer, more fluid, less polluting traffic by 2030). AI agents at the edge learn the changing radio network state and reconfigure; non-time-critical operations are offloaded further away.

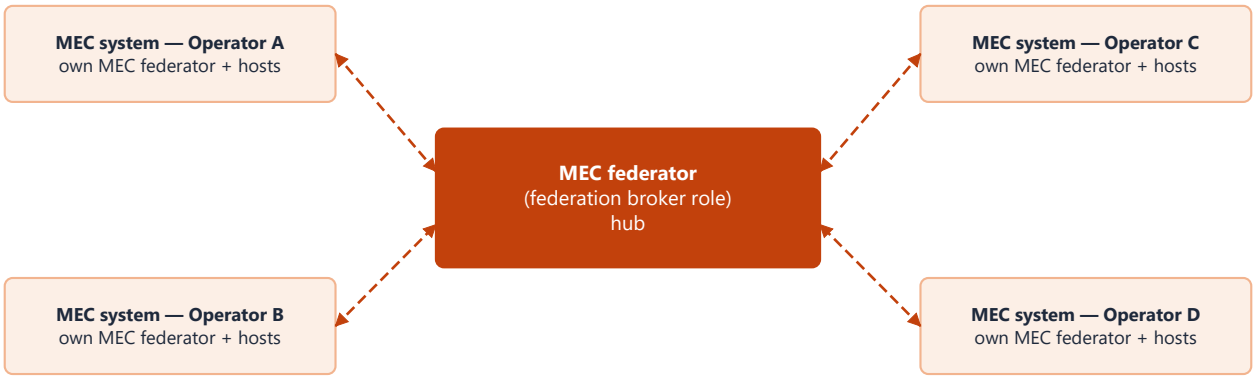
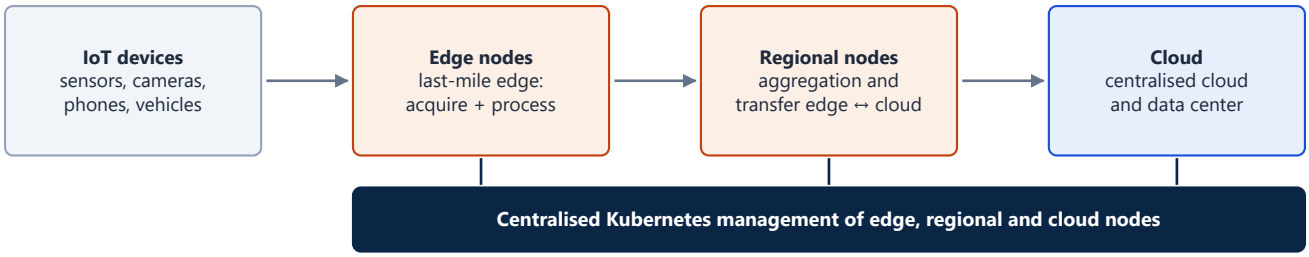


Figure 2.6 *— MEC federation: a federator acting as federation broker (hub).*

2.5 Kubernetes at the edge

Running many small sites by hand does not scale. The idea is to use the **same cloud-native tooling** (containers + Kubernetes) from the cloud down to the last-mile edge, with a centralised view of all nodes.

- **Layer 1 — cloud / data center**; **layer 2 — aggregation** (regional nodes moving data between edge and cloud); **layer 3 — last-mile edge** (acquire and process data next to the devices).
- Each edge site needs data-center qualities: **security, fault tolerance, elastic scale**.



Layer 3 (last mile) → layer 2 (aggregation) → layer 1 (cloud). Each edge site must be run like a data center: security, fault tolerance, elastic scaling.

***Figure 2.7** — Three-layer edge architecture managed with Kubernetes.*

- Execution options for a task: **local execution**, **partial offloading** or **full offloading** to a more powerful tier.
- Edge-oriented Kubernetes projects: **KubeEdge** (CNCF) and **K3s** (lightweight).

2.6 Kubernetes fundamentals

Kubernetes (K8s) is an open-source platform that automates the deployment, scaling and management of containerised applications (created by Google, now maintained by the CNCF). Key vocabulary:

Term	Meaning
Pod	Smallest deployable unit: one or more containers that share storage and network. Each Pod gets a **unique IP address**.
Node	A machine (physical or VM) that runs Pods: master (control plane) or worker.
Job controller	Runs Pods that perform a task **to completion** (batch work).
Replication controller / ReplicaSet	Keeps the requested number of identical Pods running.
Service	Stable name and IP in front of a set of Pods, with load balancing.

***Table 2.5** — Basic Kubernetes vocabulary.*

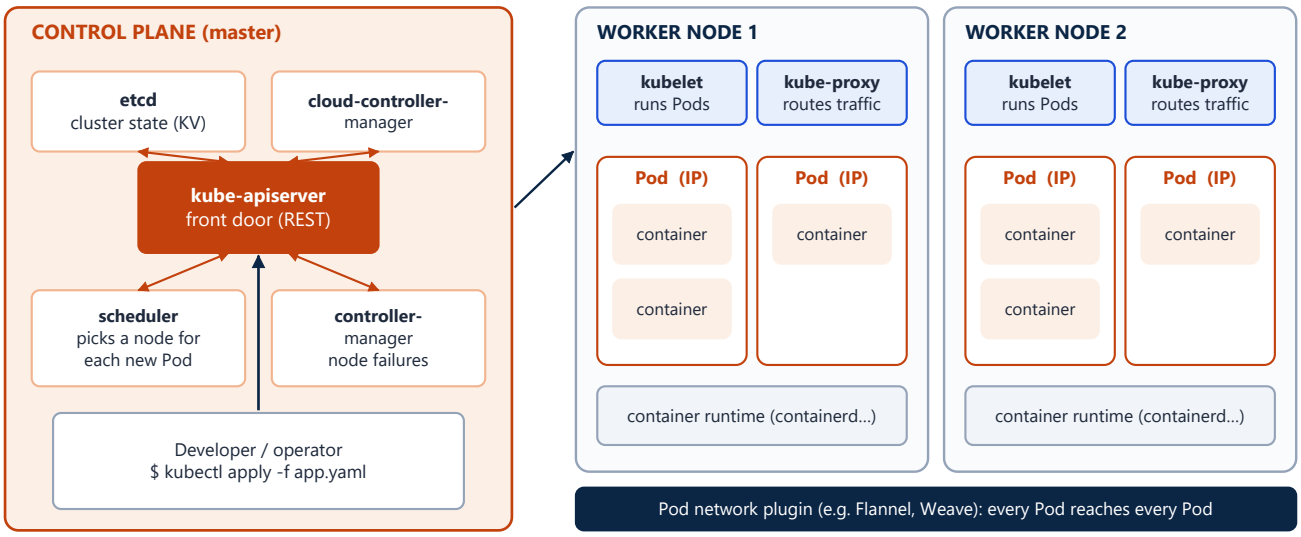


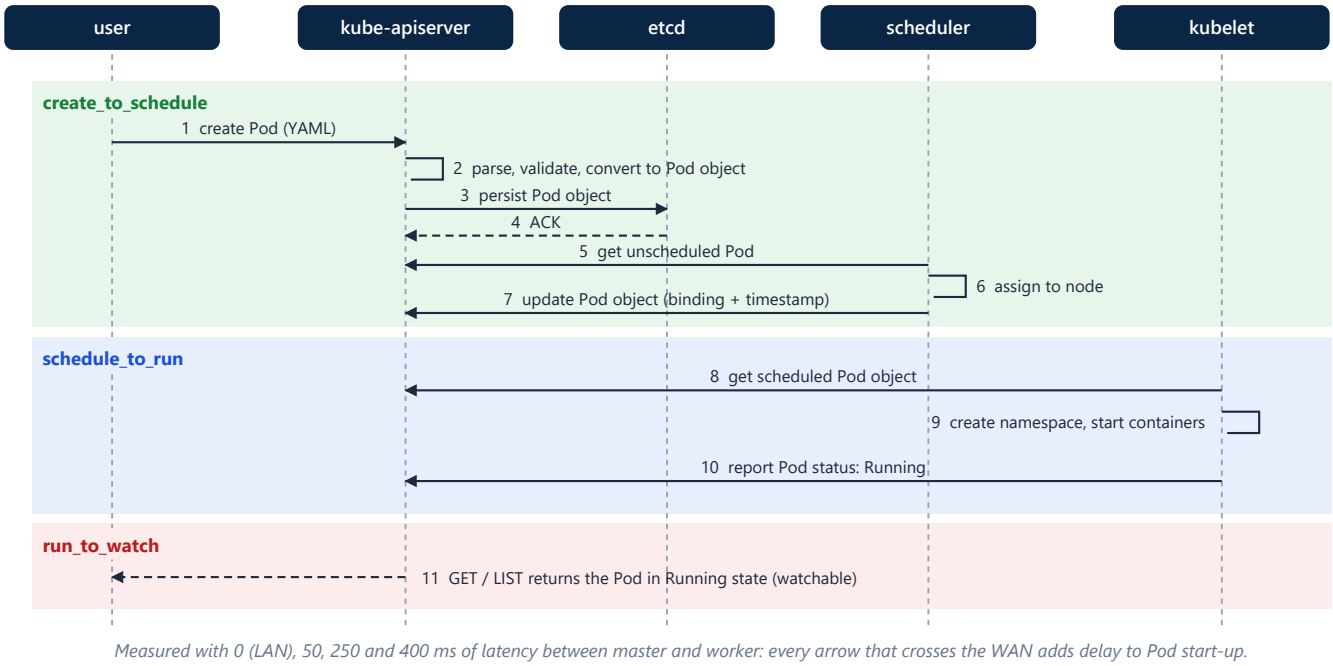
***Figure 2.8** — Kubernetes architecture: control plane and worker nodes.*

Component	Where	Role
kube-apiserver	Master	Front door of the cluster: every component and kubectl talk to it (REST API).
etcd	Master	Distributed key–value store holding the whole cluster state.
kube-scheduler	Master	Chooses a node for each newly created Pod.
kube-controller-manager	Master	Runs the controllers (node failures, replication, jobs…) that drive the cluster to the desired state.
cloud-controller-manager	Master	Links the cluster to the cloud provider's API.
kubelet	Worker	Agent that starts and watches the Pods of its node.
kube-proxy	Worker	Network rules that route Service traffic to Pods.
cAdvisor	Worker	Collects container resource usage.
Network plugin	All	Pod network such as Flannel or Weave.

Table 2.6 *— Kubernetes components.*

How a Pod starts — and why WAN latency matters

Start-up is measured in three phases: **create_to_schedule** (request accepted, stored in etcd, node chosen), **schedule_to_run** (kubelet fetches the Pod and starts its containers) and **run_to_watch** (the Running state becomes visible to the user). An Inria experiment added 0 (LAN), 50, 250 and 400 ms of latency between master and workers: since every step goes through the API server, **start-up time grows with the latency** — a key issue when the control plane is in the cloud and the workers are at the edge.



Measured with 0 (LAN), 50, 250 and 400 ms of latency between master and worker: every arrow that crosses the WAN adds delay to Pod start-up.

Figure 2.9 *— Pod start-up sequence and its three measured phases.*

From physical servers to containers

Traditional deployment runs all apps on one OS (no isolation, poor resource sharing). Virtualisation adds a hypervisor and one guest OS per VM (isolation, but heavy). Containers share the host OS kernel and package only the app and its libraries: **light, fast to start and portable** — ideal for small edge nodes.

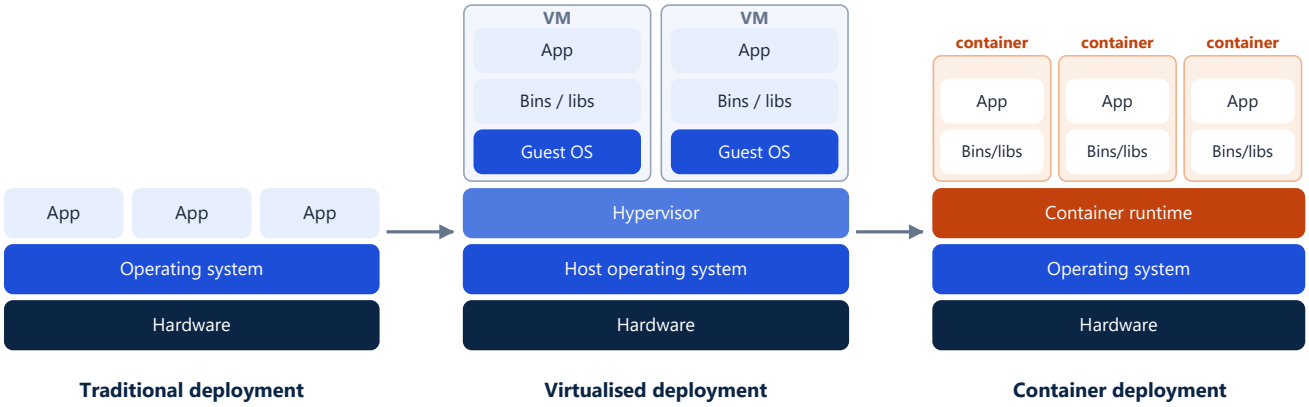


Figure 2.10 — Evolution of deployment: traditional, virtualised, containers.

Feature	What Kubernetes does
Service discovery & load balancing	Exposes containers by DNS name or IP and spreads traffic.
Storage orchestration	Mounts local or cloud storage automatically.
Automated rollouts and rollbacks	Moves to the desired state step by step; can undo.
Automatic bin packing	Places containers on nodes according to CPU / RAM needs.
Self-healing	Restarts or replaces failed containers, kills unresponsive ones.
Secret & configuration management	Stores passwords, tokens, keys separately from images.
Batch execution	Runs batch and CI workloads (Jobs).
Horizontal scaling	Scales an application up or down by command, UI or automatically.
IPv4/IPv6 dual-stack	Pods and Services can have both address families.
Designed for extensibility	New features added without changing the core code.

Table 2.7 — The ten main Kubernetes features.

2.7 Kubernetes at the edge: challenges and benefits

Challenges

- **Compute constraints**: edge nodes have limited CPU, memory and storage.
- **Remote management** of CPU, memory, network and I/O, GPUs and a distributed firewall, often without staff on site.
- **Security and trust**: device discovery, authentication, anonymity vs traceability, denial-of-service attacks.

Benefits

- **Scalability**: containers start in milliseconds, so capacity follows demand.
- **High availability**: each container has its own IP; failed Pods are replaced.
- **Deployment**: rolling updates push new versions without downtime.
- One API and toolset from cloud to edge.

2.8 Lightweight Kubernetes distributions and managed edge services

Distribution	Key points
MicroK8s (Canonical)	Minimal, production-grade K8s; immutable containers (snap); self-healing high availability; automatic promotion of nodes to datastore nodes.
K3s (Rancher → CNCF)	CNCF-certified, single binary; runs with **512 MB RAM**; Pods can run on the master; **SQLite** by default (single-master limitation unless an external DB / embedded etcd is used).
Rancher	Platform to manage **many clusters** (multi-cluster management).
KubeEdge (Huawei → CNCF)	Extends Kubernetes to edge nodes and devices; edge works offline; built-in device management (section 2.10).

Table 2.8 *— Open-source Kubernetes options for the edge.*

Managed service	Highlights
AWS IoT Greengrass	Runs Lambda functions locally, data caching, local messaging, security.
Azure IoT Edge	Built on Azure IoT Hub; deploys containers (modules) to devices; works offline.
Akamai IoT Edge Connect	Edge messaging service for connecting devices at scale.
VMware Pulse IoT Center	Management of IoT devices and edge infrastructure.
IBM Edge Application Manager	Autonomous deployment and management of edge workloads.

Table 2.9 *— Managed edge services.*

	K8s + kubeadm	K3s	MicroK8s	KubeEdge
Company	Google → CNCF	Rancher → CNCF	Canonical	Huawei → CNCF
Min. CPU (master)	2 CPUs	20 % of a Pi 4B core (BCM2711, 1.5 GHz)	—	—
Min. RAM (master)	2 GB	512 MB	4 GB	—
Min. CPU / RAM (worker)	—	10 % of a Pi 4B / 256 MB	—	—
Storage	—	< 40 MB	< 200 MB	+ 66 MB
Architectures	386, ppc64le, s390x, ARM, ARM64	x86-64, ARM, ARM64	x86-64, ARM, ARM64	x86-64, ARM, ARM64
Container runtime	containerd (default), Docker (deprecated), CRI-O	containerd (default), Docker	containerd	Docker, containerd, CRI-O, Virtlet
Ingress	—	Traefik	Nginx	—
Datastore	etcd	SQLite	etcd	etcd
OS	Linux	Linux	Linux, Windows 10, macOS (Multipass), LXD	Linux, Windows 10, macOS (Multipass)
High availability	3+ control-plane nodes	2+ servers + 0+ agents	3+ nodes	HA CloudCore

Table 2.10 *— Comparison of Kubernetes distributions (course table).*

EXAM TIP

Remember the orders of magnitude: **K8s needs 2 CPUs / 2 GB** for a master, **K3s runs in 512 MB** with SQLite, **KubeEdge adds only ~66 MB** on the edge node and its EdgeCore uses about **70 MB of memory**.

2.9 K3s in practice

K3s has two roles. A **k3s server** runs the control plane (API server, scheduler, controller manager) and can also run workloads (kubelet, kube-proxy); a **k3s agent** is a worker that runs kubelet and kube-proxy. Two topologies are presented:

Networking with Flannel. Kubernetes requires that every container can reach every other container and every node **without NAT**, and that a container sees the same IP address as the others see for it. Flannel builds an overlay network: each node receives a subnet (here 100.96.1.0/24 and 100.96.2.0/24 inside 100.96.0.0/16); the **flanneld** daemon installs routes and **encapsulates** container packets in UDP (port 8285) between the nodes' real addresses.

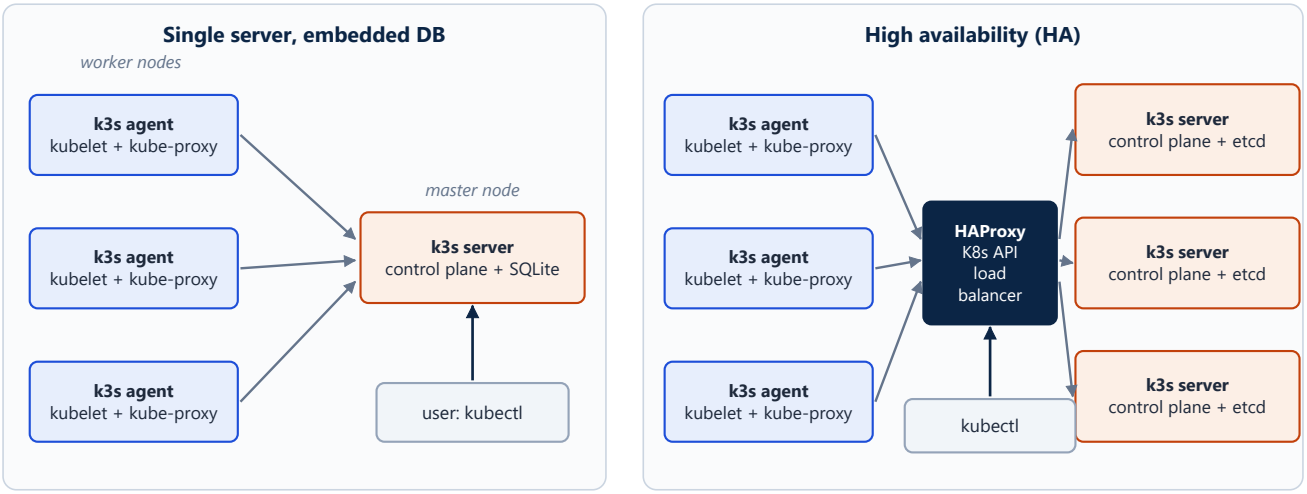


Figure 2.11 *— K3s single-server (embedded SQLite) and high-availability (3 servers behind HAProxy) topologies.*

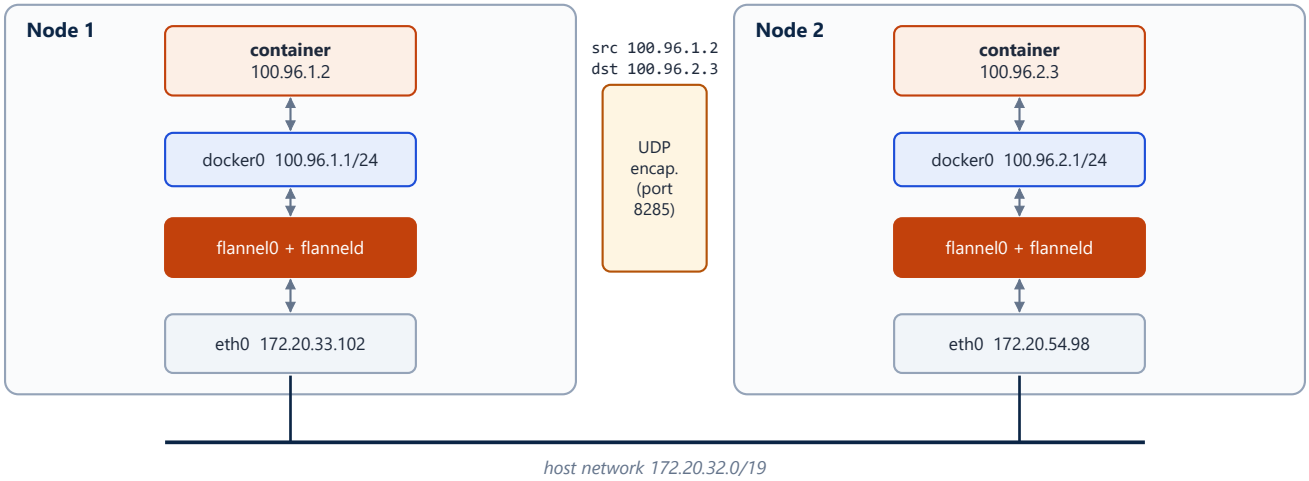


Figure 2.12 *— Flannel overlay: a packet from 100.96.1.2 (node 1) to 100.96.2.3 (node 2) is encapsulated in UDP between 172.20.33.102 and 172.20.54.98.*

For practice — the standard K3s quick-start (not on the slides):

```
# install a K3s server (standard install script)
curl -sfL https://get.k3s.io | sh -
sudo k3s kubectl get nodes
# token needed by the agents
sudo cat /var/lib/rancher/k3s/server/node-token
# join a worker (agent)
curl -sfL https://get.k3s.io | K3S_URL=https://<server-ip>:6443 K3S_TOKEN=<token> sh -
```

2.10 KubeEdge

KubeEdge (CNCF project started by Huawei) extends native Kubernetes to the edge. The cloud part, **CloudCore**, sits next to the normal Kubernetes API server; each edge node runs a light agent, **EdgeCore**. They communicate over **WebSocket** (default) or **QUIC**. It is designed for massive scale: **100,000 edge nodes and 1,000,000 Pods**, versus 5,000 nodes and 150,000 Pods for a standard Kubernetes cluster.

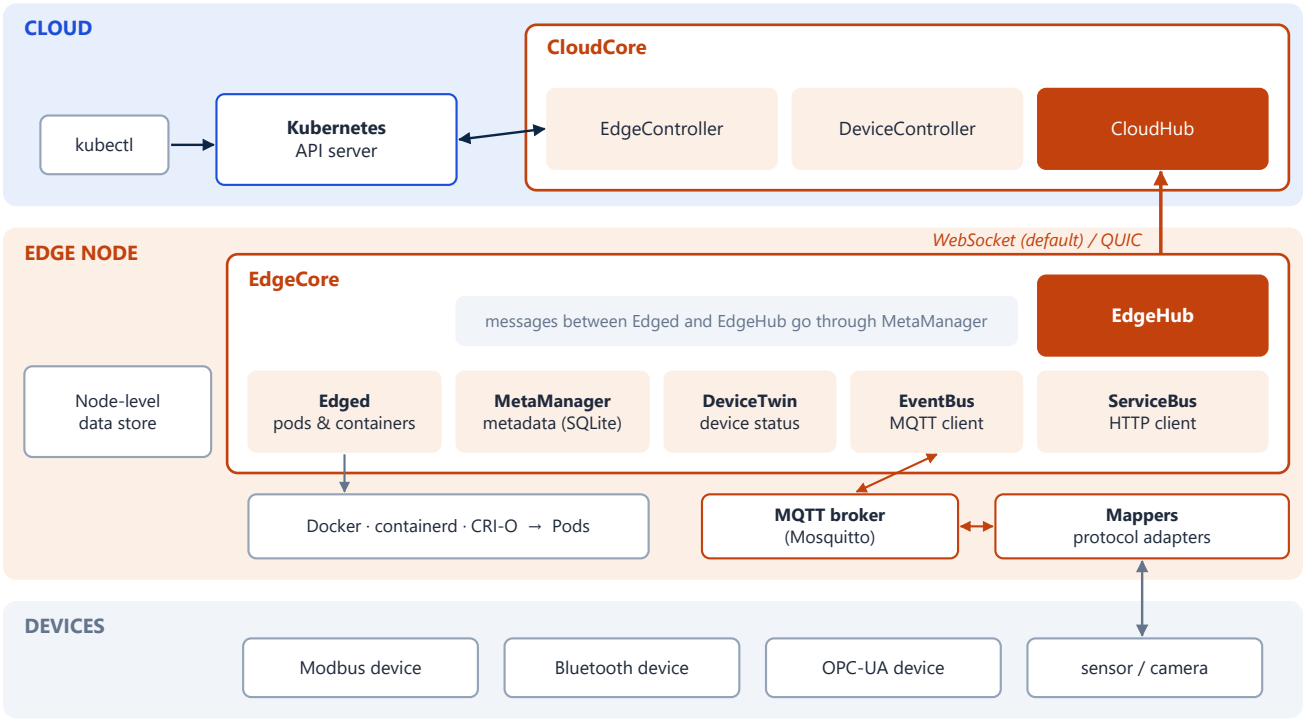


Figure 2.13 *— KubeEdge architecture: CloudCore in the cloud, EdgeCore on the edge node, mappers towards the devices.*

Component	Side	Role
CloudHub	Cloud	WebSocket server: caches and sends messages to edge nodes.
EdgeController	Cloud	Syncs Kubernetes resources (Pods, ConfigMaps…) between the API server and the edge.
DeviceController	Cloud	Syncs device metadata and status (device CRDs).
EdgeHub	Edge	WebSocket / QUIC client talking to CloudHub.
Edged	Edge	Light kubelet: runs and manages Pods and containers on the node.
MetaManager	Edge	Message processor between Edged and EdgeHub; stores metadata in **SQLite** so the node keeps working **offline**.
EventBus	Edge	MQTT client: links the edge to the MQTT broker and the devices.
ServiceBus	Edge	HTTP client: lets cloud components reach HTTP servers at the edge.

Component	Side	Role
DeviceTwin	Edge	Stores device status and syncs it with the cloud.
Mappers	Edge	Protocol adapters: Modbus, OPC-UA, Bluetooth…

Table 2.11 *— KubeEdge components.*

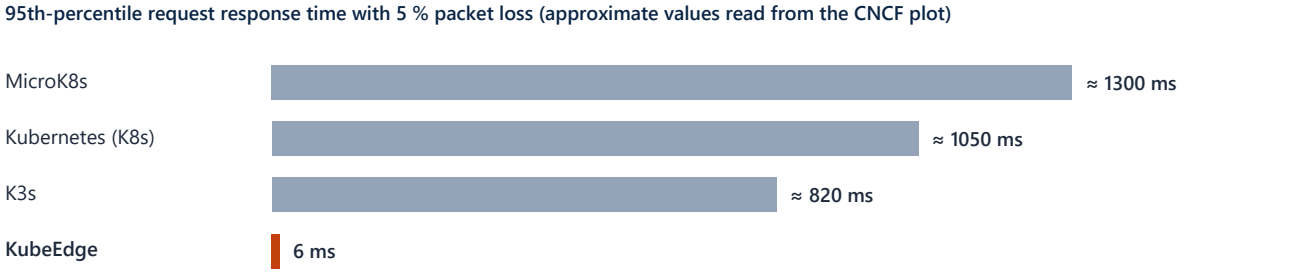
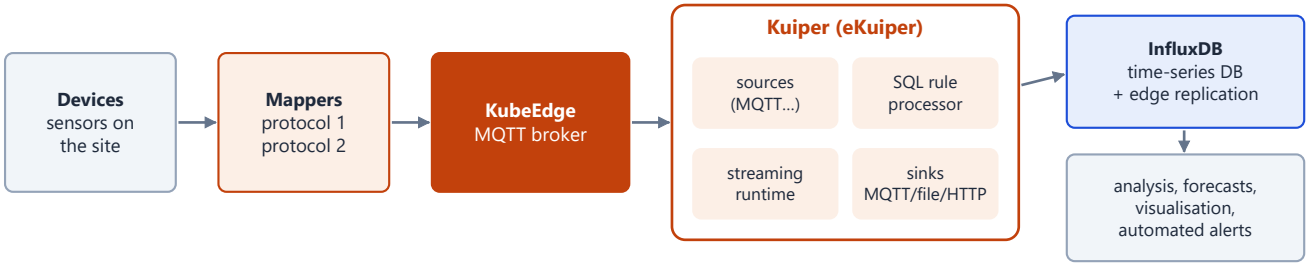


Figure 2.14 *— Request response time under 5 % packet loss: KubeEdge stays around 6 ms where the others reach about one second.*

Use cases

China Mobile — stream processing with Kuiper. Devices are read through mappers into the KubeEdge MQTT broker; **Kuiper** (eKuiper), a lightweight SQL-based stream engine, processes the streams at the edge and writes results to **InfluxDB** (with edge replication) for analysis, forecasting, visualisation and alerts. Scenarios: stream processing, a rule engine for alarms and forwarding, and format / protocol conversion.



China Mobile (Shanghai) Industrial Research Institute architecture: stream processing at the edge, storage at the edge, only results go up.

Figure 2.15 *— KubeEdge + Kuiper + InfluxDB pipeline.*

```
{
  "sql": "SELECT data->tag1->value AS temperature,
              data->tag2->value AS humidity FROM demo",
  "actions": [
    {"log": {}},
    {"mqtt": {"server": "tcp://broker.emqx.io:1883", "topic": "devices/result",
              "qos": 1, "clientId": "demo_001"}}
  ]
}
```

Sedna — edge-cloud collaborative AI. Built on KubeEdge, Sedna runs inference and training workers on edge or cloud nodes. The **Global Manager** (cloud) handles task management and coordination and model / dataset management; the **Local Controller** (edge) manages joint inference, incremental learning, federated learning and lifelong learning.

Hong Kong–Zhuhai–Macao Bridge (HZMB). KubeEdge is used to monitor the 34-mile bridge: sensors collect **14 types of data** (CO_2, light, pressure, noise, temperature…) that are processed at the edge.

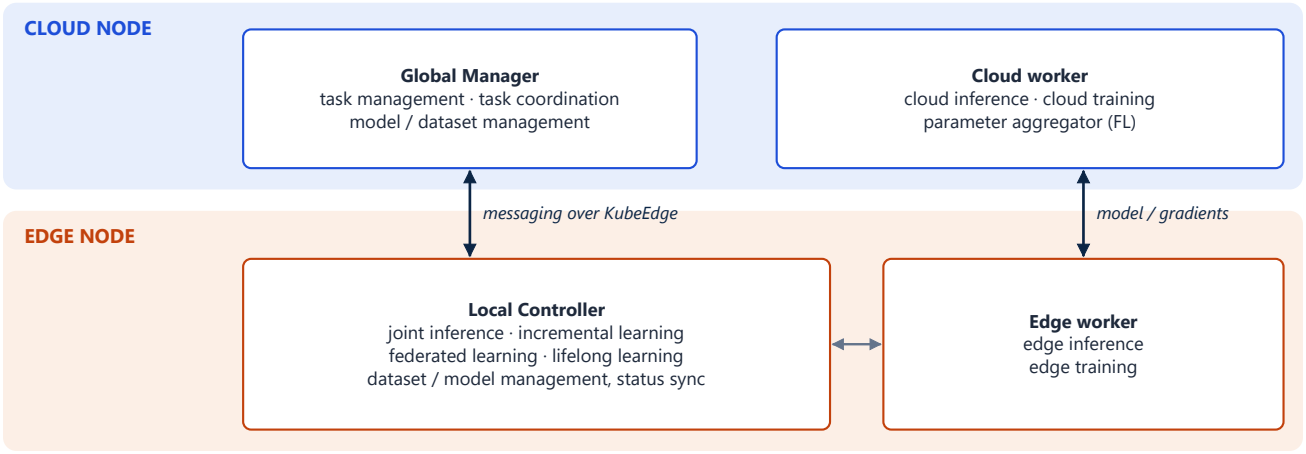


Figure 2.16 — Sedna architecture on top of KubeEdge.

2.11 The intelligent edge

An **intelligent edge** does more than forward data: it filters, analyses and acts. Seven features characterise it:

Data reduction in practice (AWS / BearingPoint). A sensor kit on each machine produces about **30 GB per month**. The edge agent turns it into operational events, generated features and asset status, so that only about **150 MB per month** per unit is sent to AWS IoT — a reduction of roughly **200×**. Octonion shows a similar idea: **anomaly detection on an ARM edge node**, with collaborative learning between nodes.



Figure 2.17 — Seven features of the intelligent edge.

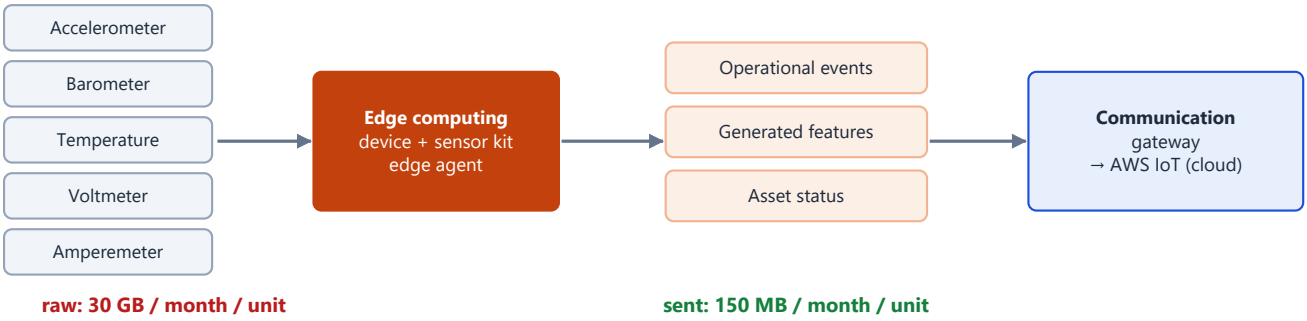


Figure 2.18 — Edge pre-processing cuts the data sent to the cloud from 30 GB to 150 MB per month and per unit.

2.12 Task offloading

Offloading means sending a task from a weak device to a more powerful edge (or cloud) server. Two questions must be answered for each task: **should** it be offloaded, and **to which** edge node? The goal is usually to minimise an expected cost (delay, energy…). The course example below shows the gain:

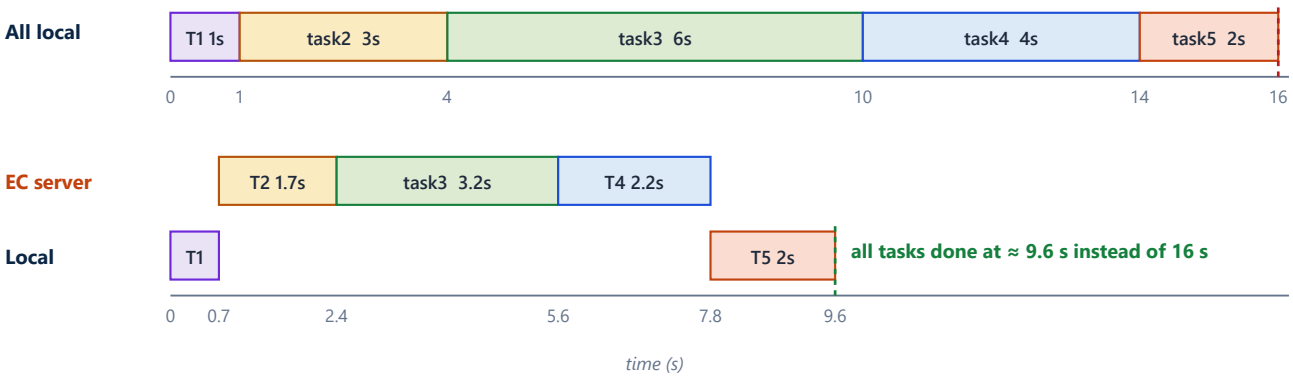


Figure 2.19 *— Five tasks executed locally (16 s) vs with tasks 2–4 offloaded to an edge-computing (EC) server (≈ 9.6 s).*

Vehicular edge computing (VEC)

In VEC, vehicles generate tasks; **road-side units (RSUs)** host MEC servers; a cloud with large capacity is reachable over optical fibre. Each vehicle has a task queue and a **scheduler** applying an **offloading policy**: run locally, on the MEC server, or in the cloud.

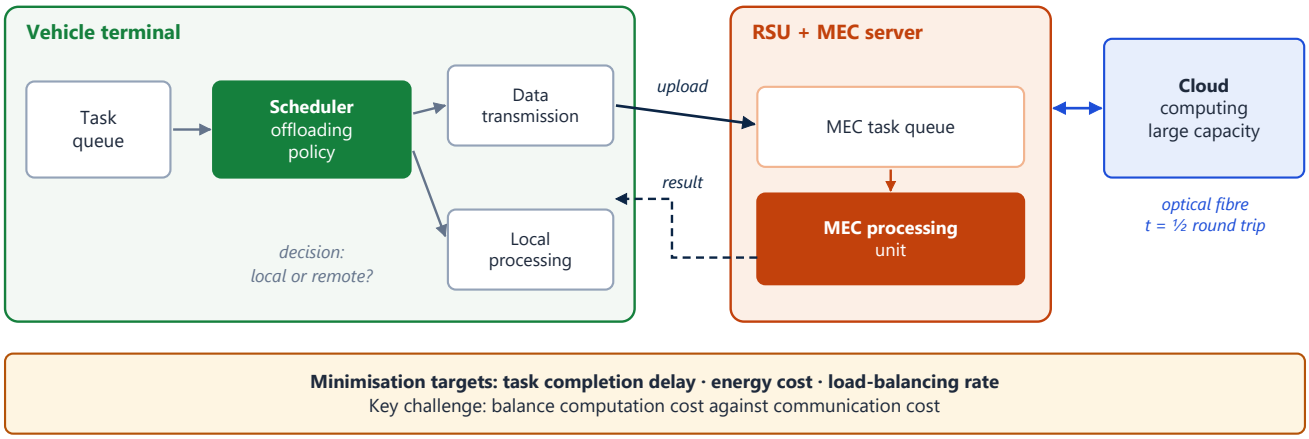


Figure 2.20 *— VEC offloading architecture (vehicle terminal, RSU + MEC server, cloud).*

Problem formulation

Minimise the average processing latency of all tasks *m* of all vehicles *k*, under the computing and communication limits of the MEC servers:

$$\min \Sigma_k \Sigma_m t_{k,m}$$

$$t_{k,m} = P^{\text{local}} \cdot t^{\text{local}} + P^{\text{edge}} \cdot t^{\text{mec}} + P^{\text{cloud}} \cdot t^{\text{cloud}}$$

$$\text{s.t. } P^{\text{local}} + P^{\text{edge}} + P^{\text{cloud}} = 1, \Sigma\, \beta \le B, \Sigma\, x \le C$$

$t_{k,m}$ = latency of task m of vehicle k; P = task location (exactly one of local / edge / cloud equals 1); β and x = allocated communication and computation resources; B and C = maximum communication and computation capability of the MEC servers.

Computation and communication models

$$t^{\text{local}} = c / f_k + t_{\text{wait}}$$

$$t^{\text{mec}} = t_{\text{trans}} + t_{\text{comp}} = d / \lambda_{k,e} + c / f_e + t_{\text{wait}}$$

$$\lambda_{k,e} = n_{k,e} \cdot w \cdot \log_2(1 + p_k \delta_{k,e} / \sigma^2)$$

$$t_{\text{off}}\,(\text{cloud}) = ½ \times \text{round-trip time MEC} \leftrightarrow \text{cloud (optical fibre)}$$

For task m of vehicle k: c = CPU cycles needed, f = CPU frequency (vehicle k or MEC server e), d = input data size, λ = uplink rate (Shannon formula) with n sub-channels of bandwidth w, transmit power p, channel gain δ and noise power σ^2.

EXAMPLE

Course example: a microcontroller at **1 MHz** (1 µs per cycle) executing a task of **1000 cycles** with no waiting time: $t^{\text{local}} = 1000 / 10^6 =$ **1 ms**.

Worked example (illustrative numbers, not from the course): a task needs $c = 0.6$ Gcycles and $d = 4$ Mbit of input. Vehicle: $f = 0.3$ Gcycles/s → $t^{\text{local}} = 2$ s. Uplink: 2 sub-channels × 1 MHz × $\log_2(1 + 15) = 8$ Mbit/s → transmission 0.5 s; MEC at 5 Gcycles/s → 0.12 s; so $t^{\text{mec}} = 0.62$ s. Cloud: 0.5 s upload + 1 s fibre (t_{off}) ≥ 1.5 s. **Best choice: the edge**, saving 1.38 s.

2.13 Reinforcement learning primer (appendix of the course)

Reinforcement learning (RL) trains an **agent** by interaction with an **environment**: at each step the agent observes a state, chooses an action, receives a **reward** and a new state, and improves its **policy** to maximise the total discounted reward. **Q-learning** is a type of RL that iteratively learns the value of each action.

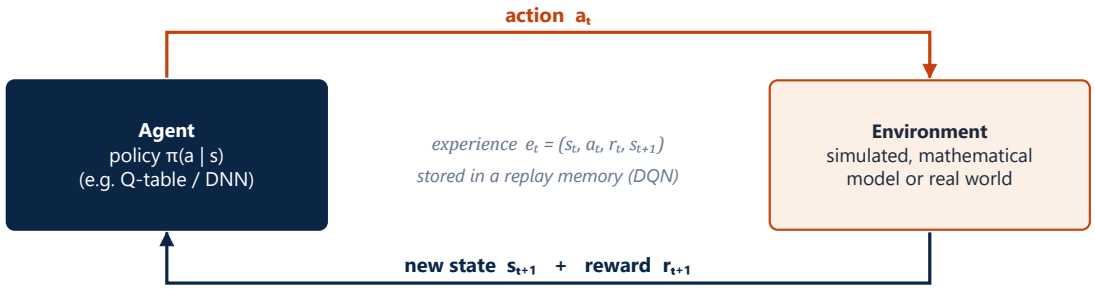


Figure 2.21 — The reinforcement-learning loop.

Concept	Meaning
Environment	Tailored simulations (robotics, game AI, driving with traffic), a **mathematical model** (as in the offloading problem), or the real world (industrial robots learning in a factory).
Reward	Feedback for each action: positive for reaching goals, negative for undesirable actions.
Policy π(a|s)	Probability of choosing action *a* in state *s*.
ε-greedy policy	Choose the best known action with probability 1 − ε, a random action with probability ε (exploration vs exploitation).
Discount factor γ	Weight of future rewards ($0 \le \gamma < 1$).
Learning rate α	Step size of each update: too low = slow, too high = unstable.

Table 2.12 — Reinforcement-learning vocabulary.

$$Q(S,A) \leftarrow Q(S,A) + \alpha\,[\,R + \gamma \max_a Q(S',a) - Q(S,A)\,]$$

Q-learning update (Bellman). Example of the course with α = 1, γ = 0.95, R = 0 and max Q(S',a) = 1: new Q = 0 + 0.95 × 1 = 0.95.

```
Initialise Q(s, a) arbitrarily; Q(terminal, .) = 0
for each episode:
    initialise S
    repeat for each step of the episode:
        choose A from S with a policy derived from Q (e.g. epsilon-greedy)
        take action A, observe R and S'
        Q(S, A) = Q(S, A) + alpha * (R + gamma * max_a Q(S', a) - Q(S, A))
        S = S'
    until S is terminal
```

From Q-table to Deep Q-Network (DQN) and Double DQN

When the state space is too large (or the episodes have no clear end), a table is impossible: a **neural network with weights θ** approximates *Q*. Training uses **experience replay**: transitions $e_t = (s_t, a_t, r_t, s_{t+1})$ are stored in a memory and sampled in random mini-batches.

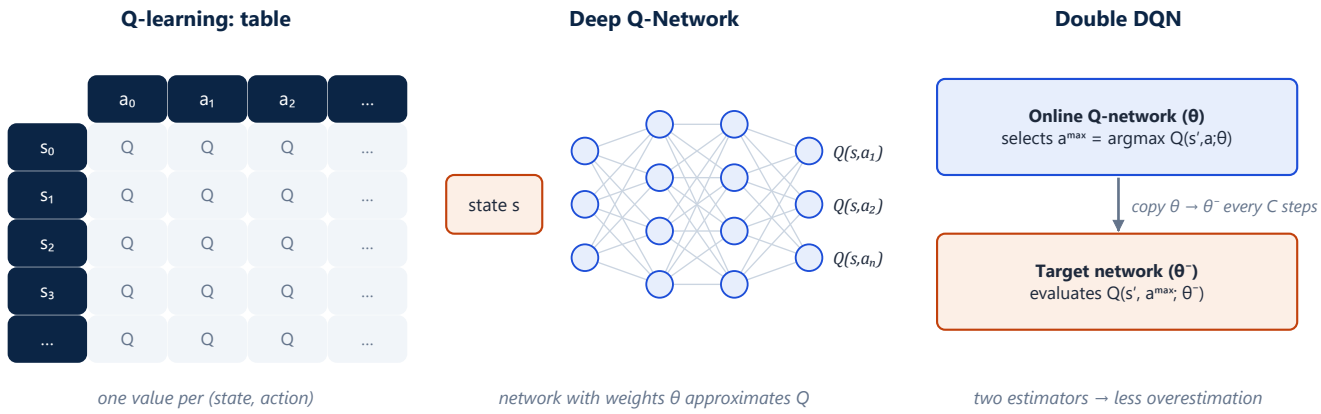


Figure 2.22 *— Q-table, Deep Q-Network and Double DQN.*

$$Q_\pi(s,a) = \mathrm{E}[\,G_t \mid s, a\,] = \mathrm{E}[\,r + \gamma\, Q_\pi(s',a')\,]$$

$$L(\theta) = \mathrm{E}[\,(y - Q(s,a;\theta))^2\,], y = r + \gamma \max_{a'} Q(s',a';\theta_{i-1})$$

$$\text{Double DQN: } a^{\max} = \operatorname{argmax}_a Q(s',a;\theta), y = r + \gamma\, Q_{\text{targ}}(s',a^{\max};\theta')$$

Plain DQN overestimates Q because the same network selects and evaluates the best action. Double DQN selects with the online network and evaluates with a target network whose parameters are copied every C iterations.

2.14 Deep Q-learning for vehicular task offloading

The offloading problem of section 2.12 is mapped onto deep Q-learning:

RL element	In the VEC offloading model
State s_t	$\{B_a, C_a\}$: communication and computation resources still available at the MEC servers.
Environment	$\{V, Q, p, \delta, B, C\}$: vehicles, task queues, transmit power, channel gains, capacities.
Action a	$\{b_{k,e}, c_{k,e}\}$: bandwidth and computation allocated to vehicle k on server e (which decides where the task runs).
Reward	0 if the task runs locally; $t^{local} - t^{mec}$ if it runs on the MEC server; $t^{local} - t^{cloud}$ if it runs in the cloud (= the time saved).

Table 2.13 *— Deep Q-learning model of the offloading problem.*

Simulation parameter	Value
Transmit power p	2 ± 0.2 W
Channel gain δ	144 ± 14.4 dB
Noise power σ^2	1.5×10^{-8} W
Vehicle CPU	0.3 ± 0.03 Gcycles/s
MEC bandwidth / CPU	40 MHz / 5 Gcycles/s
Cloud offload time t_{off}	1000 ms
Replay pool D / mini-batch	2000 / 32
Learning rate	0.01

Results.

- Learning rate **0.01** gives the highest reward (≈ 530), better than 0.005 (≈ 510) and 0.05 (≈ 470): too small learns slowly, too large oscillates.
- Average delay **grows with the number of tasks** and **falls with more MEC servers**.
- **DA-TODDPG** (dependency-aware DDPG) is best everywhere, then DA-TODQN (DQN), E-TODDPG and the random baseline DA-RO.

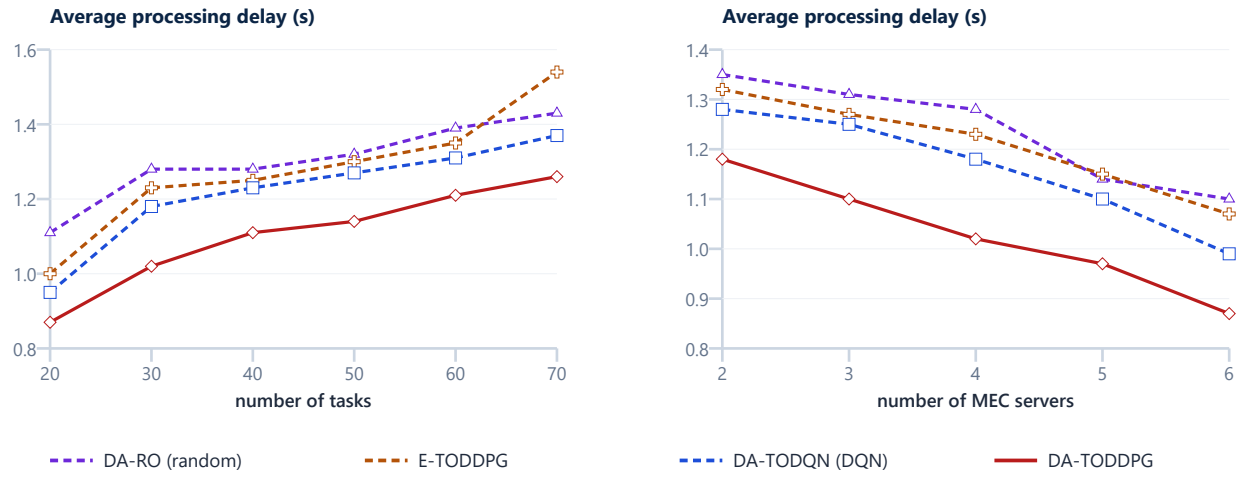


Figure 2.23 *— Average processing delay vs number of tasks (left) and vs number of MEC servers (right).*

2.15 Queueing models for edge and fog

Queueing theory predicts waiting times when jobs arrive at random at servers of limited speed. In Kendall notation **M/M/1** means Poisson arrivals (rate λ), exponential service times (rate μ) and one server; **M/M/N** has N parallel servers.

$$\rho = \lambda / \mu < 1,\ L = \rho / (1-\rho),\ W = 1 / (\mu - \lambda),\ L = \lambda W \ \text{(Little)}$$

M/M/1 reminder (standard results, not on the slides). Example: λ = 8 jobs/s, μ = 10 jobs/s → ρ = 0.8, W = 0.5 s, L = 4 jobs. At ρ = 0.9, W doubles to 1 s: queues explode near saturation.

Single-server edge model (Tang & Wong, 2022)

Time is slotted (Δ = 0.1 s). Each of the M mobile devices receives tasks $k_m(t)$ and decides $x_m(t) \in \{0,1\}$: compute locally (computation queue served by its CPU f^{device}) or offload (transmission queue over the wireless link to edge node n, chosen by $y_{m,n}(t) \in \{0,1\}$). Each of the N edge nodes keeps M queues; the $B_n(t)$ active ones share its CPU f^{edge}.

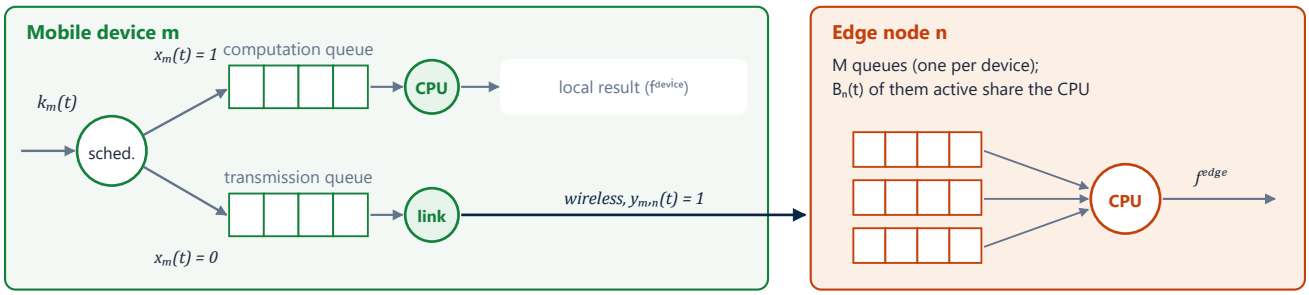


Figure 2.24 — Queueing model of a mobile device and an edge node.

Cloud–fog queueing network

Client requests (each client server CS is M/M/1) reach the **fog servers** (M/M/N). With probability 1 − κ the fog answers directly; with probability κ it lacks power and forwards the job to the **cloud**: entry server ES (M/M/1) → processing servers PS (M/M/R) → database server DS (M/M/1, visited with probability δ) → output server OS (M/M/1, reached with probability τ) → back to the client.

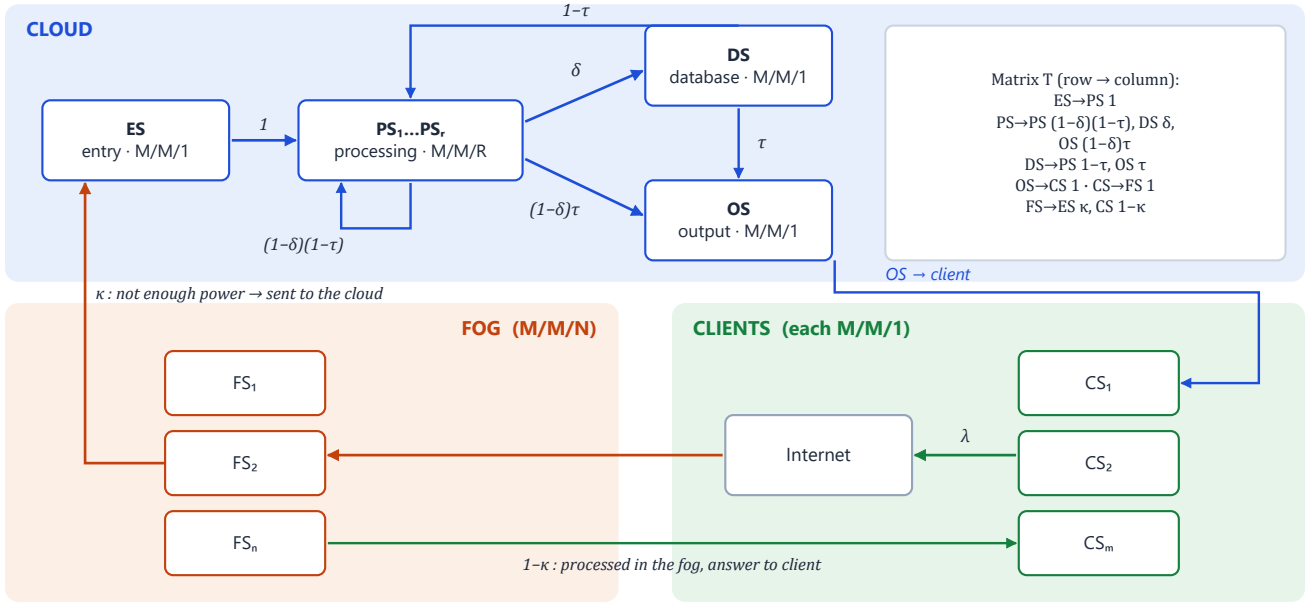


Figure 2.25 — Cloud–fog queueing network and its routing probabilities.

from \ to	ES	PS	DS	OS	CS	FS
ES	0	1	0	0	0	0
PS	0	$(1-\delta)(1-\tau)$	δ	$(1-\delta)\tau$	0	0
DS	0	$1-\tau$	0	τ	0	0
OS	0	0	0	0	1	0
CS	0	0	0	0	0	1
FS	κ	0	0	0	$1-\kappa$	0

Table 2.14 *— Routing (transition) matrix of the cloud–fog model. Other parameters: service rates μ^E, μ^P, μ^D, μ^O, μ^F, μ^C; R, N, M servers; J jobs in the network.*

A similar model for the Internet of Vehicles (Wei & Liang, 2024) sends each request to the edge server with probability p, or over the transmission link to the cloud with probability $1 - p$; optimising p balances edge congestion against transmission delay.

2.16 RL-based Kubernetes scheduling

The default Kubernetes scheduler places Pods with fixed rules. A research prototype (Politecnico di Torino thesis) replaces the scoring step with **reinforcement learning**: **Prometheus** collects node metrics, an **RL model scores each node**, and a **scheduler plugin** queries it through suggestion APIs before placing latency-sensitive applications on the worker nodes of a **Kind** cluster.

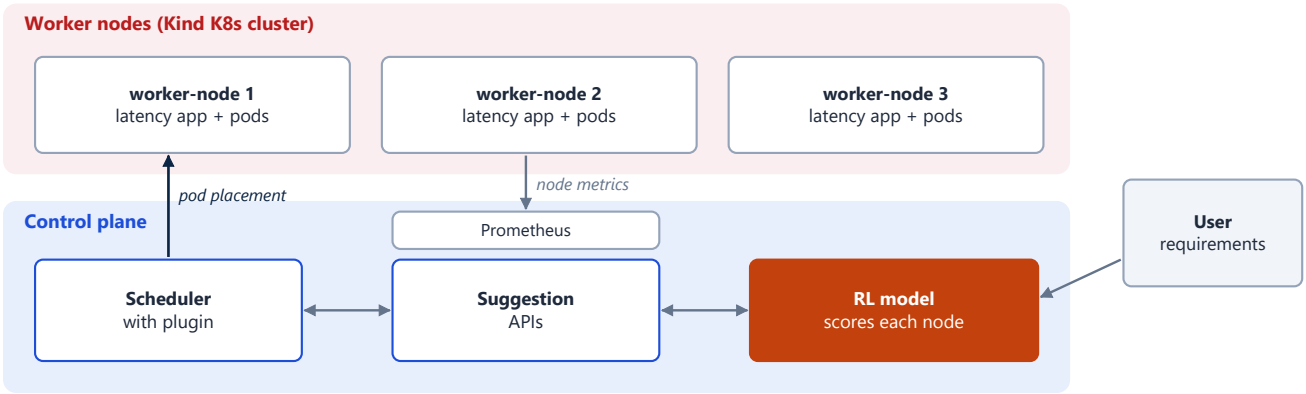


Figure 2.26 *— RL-based scheduler plugin for Kubernetes.*

2.17 Chapter 2 recap and review questions

KEY IDEA · KEY POINTS

- Edge = processing near the data: latency (< 20 ms vs up to 250 ms), privacy, bandwidth.
- Continuum: cloud (10+ ms) → thick edge (1–10 ms) → thin edge (0.1–1 ms) → micro edge (< 0.1 ms).
- ETSI MEC: orchestrator + OSS (system level), platform manager + VIM (host level), MEC host = platform + apps + NFVI; federation through a MEC federator.
- Kubernetes: API server, etcd, scheduler, controller manager; kubelet, kube-proxy, Pods.
- Edge distributions: K3s (512 MB, SQLite), MicroK8s, KubeEdge (CloudCore / EdgeCore, offline, 100k nodes).
- Offloading: compare $t^{local} = c/f$ with $t^{mec} = d/\lambda + c/f_e$; solved with DQN / DDPG; DA-TODDPG best.
- Queueing: M/M/1 for single servers, M/M/N for the fog, routing probabilities δ, τ, κ.

Q1 Give three reasons to process IoT data at the edge rather than in the cloud.

Latency (cloud round trip up to 250 ms vs under 20 ms at the edge), data privacy (sensitive data stays local), bandwidth and cost (only aggregates are sent), plus offline reliability.

Q2 What are the latency and distance of a thin edge compared with the cloud?

Thin edge: 0.001–1 km, 0.1–1 ms. Cloud: 10+ km, 10+ ms.

Q3 What does the multi-access edge orchestrator do, and what does the MEC platform manager do?

The orchestrator works at system level: it selects MEC hosts according to latency, resources and services and keeps a global view. The platform manager works at host level: application life cycle, rules, authorisations, traffic rules, DNS and conflicts.

Q4 Why can a Kubernetes control plane in the cloud slow down Pods at the edge?

Every step of Pod creation goes through the API server (store in etcd, scheduling, kubelet fetch, status report). Each WAN crossing adds latency, so start-up time grows with master–worker latency (tests at 50, 250, 400 ms).

Q5 Why does a KubeEdge node keep working when the cloud link is lost?

MetaManager stores the metadata in a local SQLite database and Edged runs the Pods locally, so the node can restart and manage its workloads offline; EdgeHub resyncs with CloudHub when the link returns.

Q6 Name the K3s roles and its default datastore. How is K3s made highly available?

k3s server (control plane, can run workloads) and k3s agent (worker). Default datastore: SQLite. HA: several servers (e.g. 3 with etcd) behind a load balancer such as HAProxy.

Q7 Write the local and MEC completion times of a task.

$t^{local} = c/f_k + wait$; $t^{mec} = d/\lambda + c/f_e + wait$, with $\lambda = n \cdot w \cdot \log_2(1 + p \cdot \delta/\sigma^2)$.

Q8 What reward does the DQL offloading agent receive?

0 for local execution, $t^{local} - t^{mec}$ for MEC execution, $t^{local} - t^{cloud}$ for cloud execution: the time saved.

Q9 What problem does Double DQN solve?

Overestimation of Q-values: the online network selects the best next action and a separate target network (updated every C steps) evaluates it.

Q10 In the cloud–fog model, what do κ and δ represent?

κ: probability that a fog server forwards the job to the cloud (not enough power); δ: probability that a processed job must access the database server.

Cheat sheet: the numbers and lists to know

One page to reread just before the exam. Teal = Chapter 1, orange = Chapter 2.

Chapter 1 · IoT

Item	Value
IoT revenue	$181.5 B (2020) → $621.6 B (2030, forecast)
Economic impact	$4–11 trillion per year by 2025
Layers	Device · Edge / fog · Network · Cloud
World Forum models	D2D · D2C · D2G2C · D2G2C2App
CoAP device	10 KB RAM · 100 KB storage

Technology	Rate · range · battery
ZigBee (802.15.4)	250 kbit/s · ≤ 100 m · years
Bluetooth (802.15.1)	720 kbit/s · 10–100 m · ≈ 1 week
Wi-Fi (802.11)	≤ 54 Mbit/s · 100 m+ · hours
LPWAN (LoRaWAN, Sigfox, NB-IoT)	low bit rate · 5–40 km

	MQTT	CoAP
Transport	TCP (1883 / 8883 TLS)	UDP (5683)
Model	publish / subscribe via broker	request / response (REST)
Operations	publish, subscribe to topics	GET, POST, PUT, DELETE
Reliability	QoS 0 / 1 / 2	CON + ACK, NON

Chapter 2 · Edge

Item	Value
Cloud round trip	up to 250 ms (edge < 20 ms)
Continuum latency	cloud 10+ ms · thick 1–10 · thin 0.1–1 · micro < 0.1
Edge market	$11.6 B (2019) → $30.8 B (2025)
AWS / BearingPoint	30 GB → 150 MB per month
Pod start-up phases	create_to_schedule · schedule_to_run · run_to_watch

Distribution	Footprint
Kubernetes master	2 CPUs · 2 GB · etcd
K3s	512 MB · < 40 MB · SQLite · Traefik
MicroK8s	4 GB · < 200 MB · Nginx
KubeEdge	+66 MB · EdgeCore ≈ 70 MB RAM · 100k nodes

Formula	Meaning
$t^{local} = c / f$	cycles ÷ CPU frequency
$t^{mec} = d/\lambda + c/f_e$	upload + edge compute
$\lambda = n \cdot w \cdot \log_2(1 + p\delta/\sigma^2)$	uplink rate (Shannon)
$Q \leftarrow Q + \alpha[R + \gamma \max Q' - Q]$	Q-learning update
$W = 1/(\mu - \lambda)$, $\rho = \lambda/\mu$	M/M/1 time in system, load

REMEMBER · LAST-MINUTE LIST

- **MEC** = ETSI standard, built on **NFV**, aligned with **3GPP / 5G**; orchestrator + OSS (system) · platform manager + VIM (host).
- **KubeEdge**: CloudCore (CloudHub, EdgeController, DeviceController) ↔ EdgeCore (EdgeHub, Edged, MetaManager, EventBus, ServiceBus, DeviceTwin) over WebSocket / QUIC.
- **Offloading reward** = time saved; **DA-TODDPG** gives the lowest delay; best learning rate 0.01.
- **Fog queueing**: clients M/M/1, fog M/M/N, cloud ES → PS (M/M/R) → DS → OS; κ = sent to cloud, δ = database access, τ = output.

NOTE · ABOUT THIS DOCUMENT

This summary follows the structure of the two course decks by Dr. Sofiane Ouni (2026). Paragraphs marked *beyond the slides* and the worked numeric examples were added to help understanding; everything else comes from the course. The DS 2022 correction at the end is a proposed correction, not the teacher's answer key.

Glossary of acronyms

All abbreviations used in the two chapters, in alphabetical order.

Acronym	Meaning
3GPP	3rd Generation Partnership Project (mobile standards: 4G, 5G)
5G	Fifth generation of mobile networks
6LoWPAN	IPv6 over Low-Power Wireless Personal Area Networks (IEEE 802.15.4)
ACK / CON	Acknowledgement / Confirmable message (CoAP)
API	Application Programming Interface
AR / VR	Augmented / Virtual Reality
BLE	Bluetooth Low Energy
BPL / NB-PLC	Broadband over Power Lines / Narrowband Power-Line Communication
CDN	Content Delivery Network
CNCF	Cloud Native Computing Foundation
CoAP	Constrained Application Protocol
CRI-O	Container runtime implementing the Kubernetes Container Runtime Interface
CSP	Cloud Service Provider
D2D / D2C / D2G2C	Device-to-Device / -to-Cloud / -to-Gateway-to-Cloud
DDPG	Deep Deterministic Policy Gradient
DNS	Domain Name System
DQN / DQL	Deep Q-Network / Deep Q-Learning
DTLS	Datagram Transport Layer Security (TLS for UDP)
EC	Edge Computing (server)
ETSI	European Telecommunications Standards Institute
HA	High Availability
HTTP / REST	Hypertext Transfer Protocol / Representational State Transfer
HZMB	Hong Kong–Zhuhai–Macao Bridge
IEEE	Institute of Electrical and Electronics Engineers
IoFT	Internet of Flying Things (drones in the IoT)
IoT / IIoT	Internet of Things / Industrial IoT
IP / IPv6	Internet Protocol (version 6)
JSON	JavaScript Object Notation (text data format)
K8s / K3s	Kubernetes / lightweight Kubernetes distribution by Rancher
LAN / MAN / WAN	Local / Metropolitan / Wide Area Network
LoRaWAN	Long Range Wide Area Network
LPWAN	Low-Power Wide-Area Network

Acronym	Meaning
M2M	Machine-to-Machine
MEC	Multi-access Edge Computing (ETSI)
ML / AI	Machine Learning / Artificial Intelligence
M/M/1, M/M/N	Queue with Poisson arrivals, exponential service, 1 or N servers
MQTT	Message Queuing Telemetry Transport
MQTT-SN	MQTT for Sensor Networks (runs over UDP / ZigBee)
NAT	Network Address Translation
NB-IoT	Narrowband IoT (cellular LPWAN)
NFV / NFVI	Network Functions Virtualisation / NFV Infrastructure
OSCORE	Object Security for Constrained RESTful Environments
OSS	Operations Support System
PoE	Power over Ethernet
QoS	Quality of Service
QUIC	UDP-based secure transport protocol (used by HTTP/3)
RAN	Radio Access Network
RFID	Radio-Frequency Identification
RL	Reinforcement Learning
RSU	Road-Side Unit
SDK	Software Development Kit
SQL / SQLite	Structured Query Language / embedded SQL database
TCP / UDP	Transmission Control Protocol / User Datagram Protocol
TLS	Transport Layer Security
TPM	Trusted Platform Module (security chip)
UAV	Unmanned Aerial Vehicle (drone)
UE	User Equipment
V2X	Vehicle-to-Everything communication
VEC	Vehicular Edge Computing
VIM	Virtualisation Infrastructure Manager
VM / VPC	Virtual Machine / Virtual Private Cloud
WSN	Wireless Sensor Network
ZigBee	Low-power mesh protocol on IEEE 802.15.4

DS 2022 — proposed correction

Technologies et Protocoles pour l'IoT · RT5 · 1 h 30 · documents allowed · 2 pages · 10 + 10 points.

Exercise 1 checks the application protocols of Chapter 1 (MQTT, CoAP, REST) plus **MQTT-SN**, the MQTT variant for sensor networks. **Exercise 2** applies MQTT, then REST, to a fleet of drones driven by a Raspberry Pi used as an **edge node**, the idea of Chapter 2. Each question is restated in English, then answered.

NOTE · ABOUT THIS CORRECTION

This is a **proposed correction** written for this summary, not the teacher's official answer key. Some answers use standard protocol facts that are not on the slides (MQTT QoS exchanges, MQTT-SN, CoAP Observe from RFC 7641). The code targets Python 3 with paho-mqtt, requests and Flask; IP addresses are examples.

Exercise 1 MQTT, CoAP, REST and MQTT-SN (10 points)

Q1 *Copy the table and give the characteristics and the differences of MQTT, CoAP, REST and MQTT-SN: TCP/UDP connection, type of communication (client–server…), point-to-point or multipoint (concentrator) link, security, packet size, and complexity in number of messages.*

Answer. There are two families. MQTT and MQTT-SN **push** data through an intermediary (publish / subscribe). REST and CoAP **pull** data from a server (request / response); CoAP is the light UDP version of REST.

	MQTT	CoAP	REST (HTTP)	MQTT-SN
Connection TCP / UDP	**TCP**, port 1883 (8883 with TLS); persistent connection to the broker	**UDP**, port 5683 (5684 with DTLS); no connection	**TCP** (HTTP over TCP), port 80 / 443	**UDP** or any datagram link (ZigBee, BLE); no TCP needed
Type of communication	**Publish / subscribe** through a broker; asynchronous, event-driven	**Client / server**, request / response (GET, POST, PUT, DELETE) + Observe for notifications	**Client / server**, request / response, stateless (GET, POST, PUT, DELETE)	**Publish / subscribe** through a gateway that forwards to an MQTT broker
Point-to-point or multipoint	**Multipoint**, many-to-many; the broker is the concentrator	**Point-to-point** (unicast); also IP **multicast** for groups and discovery	**Point-to-point**: one client ↔ one server	**Multipoint**: the gateway concentrates the sensors; broadcast to find a gateway
Security	**TLS** + username / password, access rules (ACL) on the broker	**DTLS** (coaps://); OSCORE	**HTTPS** (TLS) + tokens, API keys, OAuth	None in the protocol: link security (ZigBee AES-128) or DTLS
Packet size	Small: 2-byte fixed header, but the topic name travels in every PUBLISH	Small: 4-byte binary header, one UDP datagram	**Large**: text HTTP headers (hundreds of bytes) + JSON / XML body	**Smallest**: 2-byte header, 2-byte topic ID instead of the name; fits a 127-byte 802.15.4 frame
Complexity (number of messages)	Medium: TCP handshake + CONNECT / CONNACK, then 1, 2 or 4 messages per PUBLISH (QoS 0 / 1 / 2) + keep-alive pings	**Low**: 1 request + 1 response (ACK carries the answer); NON = 1 message; no set-up	**High**: TCP (+ TLS) handshake, request + response for each exchange; the client must poll	**Low**: no TCP; REGISTER / REGACK once per topic, then PUBLISH; QoS −1 sends without connecting

Table D.1 *— MQTT, CoAP, REST and MQTT-SN compared (answer to Q1).*

EXAM TIP

Fast way to fill the table: **TCP** → MQTT, REST · **UDP** → CoAP, MQTT-SN. **Broker / gateway** → MQTT, MQTT-SN (multipoint, push) · **client / server** → REST, CoAP (point-to-point, pull; CoAP can push with Observe). Overhead grows roughly MQTT-SN ≈ CoAP < MQTT < REST.

Q2 *Several subscribers are subscribed to the same topic M. Which QoS should be used to get a confirmation that all the subscribers received the message? Draw a diagram of the solution.*

Answer: QoS 1 (at least once), or **QoS 2** (exactly once) if a subscriber must never get M twice. It must be used on **both legs**: the publisher publishes with QoS ≥ 1 **and** every subscriber subscribes to M with QoS ≥ 1. A subscriber receives min(QoS of the publish, QoS of its subscription), so one subscription at QoS 0 removes the confirmation for that subscriber.

MQTT acknowledges **hop by hop**. The broker answers the publisher with PUBACK, then sends one PUBLISH to each subscriber, and each subscriber answers with its own PUBACK. Until it gets that PUBACK the broker sends the message again (DUP flag). For a subscriber with a persistent session (clean session = 0), the broker even keeps the message while the subscriber is offline.

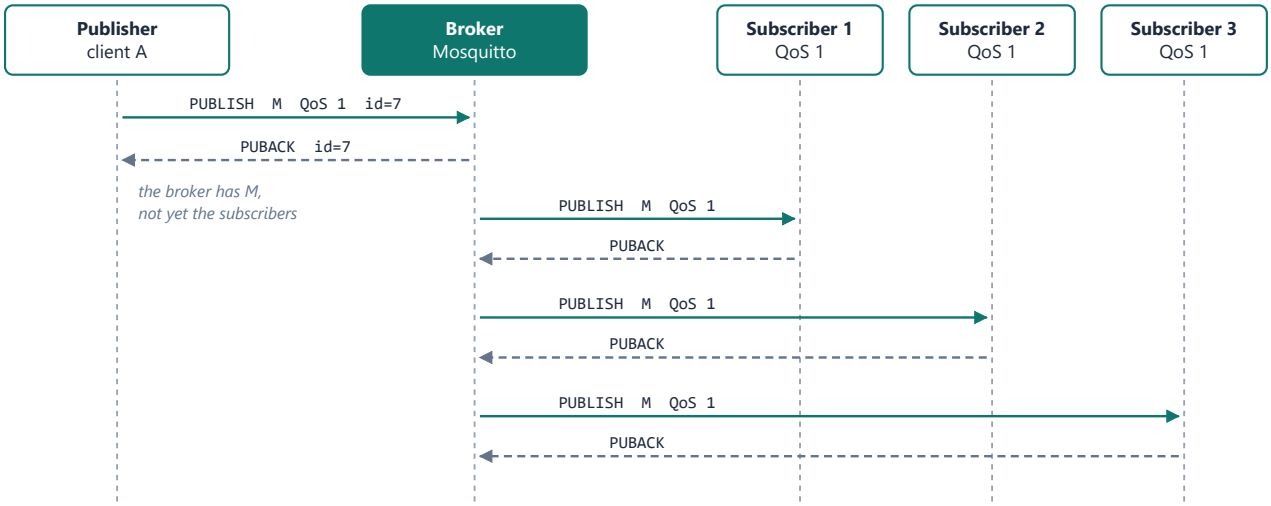


Figure D.1 *— QoS 1 with three subscribers: every hop has its own PUBACK. With QoS 2, each PUBACK is replaced by the 4-step exchange PUBLISH → PUBREC → PUBREL → PUBCOMP.*

EXAM TIP

The PUBACK received by the publisher only proves that the **broker** has M, not that the subscribers have it. If the publisher itself must know that every subscriber got M, add an **application acknowledgement**: each subscriber publishes a short message on a reply topic such as M/ack. This is exactly the role of the */processed* topic in Exercise 2.

Q3 *A microcontroller board carries a sensor that reads the atmospheric pressure. The board uses a WSN (Wireless Sensor Network) such as ZigBee, which does not use TCP. The pressure values must reach an MQTT broker and then be used by a control computer. Propose an architecture as a diagram showing the components and their interactions.*

Answer. MQTT runs over TCP/IP, which the ZigBee node does not have, so the node cannot talk to the broker directly. The standard solution is **MQTT-SN** (MQTT for Sensor Networks): the node runs a small MQTT-SN client over ZigBee, and a **gateway** at the border of the WSN translates MQTT-SN into MQTT over TCP/IP. This is the device → gateway → cloud model of Chapter 1.

- **Sensor node**: microcontroller + pressure sensor + ZigBee radio (e.g. an XBee module). It publishes with MQTT-SN, using a 2-byte topic ID to save bytes.
- **Gateway** (e.g. a Raspberry Pi with a ZigBee coordinator): ZigBee on one side, Wi-Fi / Ethernet on the other. It runs an MQTT-SN gateway (e.g. the Eclipse Paho MQTT-SN gateway) and is an MQTT client of the broker.
- **MQTT broker** (e.g. Mosquitto): receives station/pressure and pushes it to the subscribers.

- **Control computer**: MQTT client subscribed to station/pressure; it displays and stores the values and raises alarms.

Simpler alternative: the node sends raw ZigBee frames, and a program on the gateway reads them and publishes with MQTT itself. It works, but every new sensor needs code on the gateway, while MQTT-SN keeps the topics from end to end.

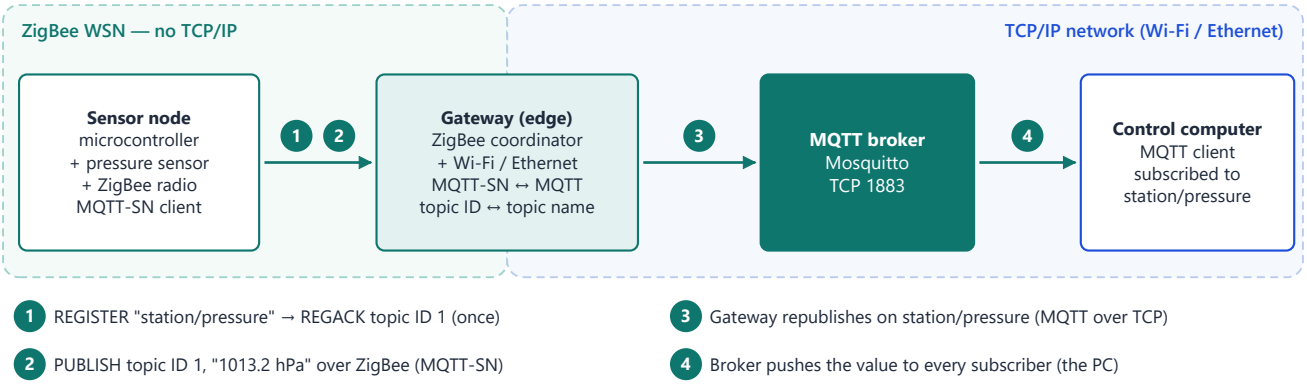


Figure D.2 *— MQTT-SN architecture: the gateway sits on both networks and turns topic IDs back into topic names.*

> **Q4** *Same question as before, but using the REST protocol instead of MQTT. Propose a solution architecture with a diagram.*

Answer. REST works over HTTP, so over TCP/IP as well: the gateway is still needed. It reads the ZigBee frames and acts as an **HTTP client** that POSTs each measurement to a **REST server** (for example a Flask API with a database). The control computer is another HTTP client that reads the value with GET.

The difference with MQTT: the exchange is **request / response**, so data never arrives by itself. The computer must **poll** (a GET every few seconds) or the server must call it back (webhook), and each value costs a full HTTP request and response with text headers. A lighter variant gives the server role to the gateway itself (GET /pressure on the gateway). To run REST *on the node*, use CoAP over 6LoWPAN (IPv6 over 802.15.4) with an HTTP ↔ CoAP proxy on the gateway.

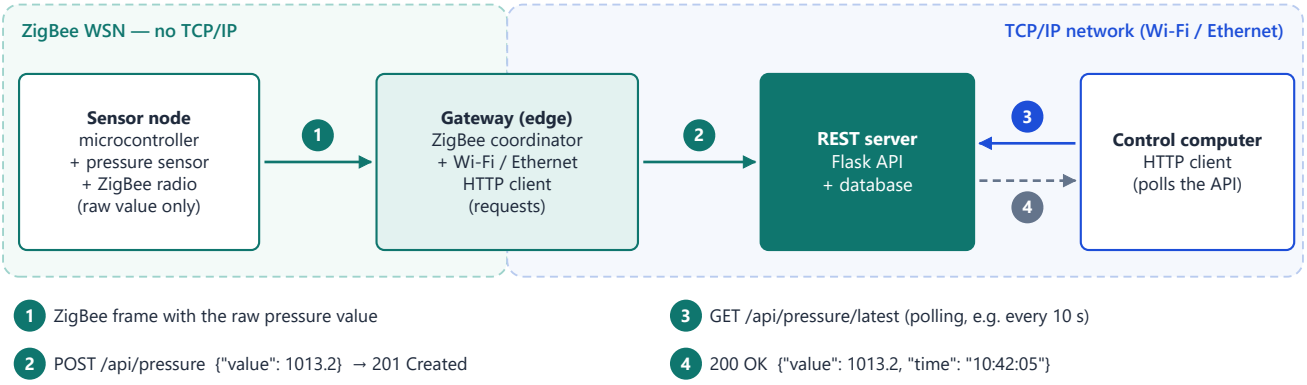


Figure D.3 *— REST architecture: no broker; a server keeps the resource and the control computer has to ask for it.*

Q5 *How can CoAP deliver data automatically, as MQTT does for its subscribers? Give an example with an explanatory diagram.*

Answer: the CoAP Observe option (RFC 7641). The client sends a GET on the resource with **Observe: 0**; the server adds it to the list of observers of that resource and answers with the current value. After that, each time the value changes, the server sends a new **2.05 Content** notification with the same token and a growing Observe number, without any new request. Notifications are CON (acknowledged) or NON. The client stops with a GET carrying **Observe: 1**, or by answering a notification with RST.

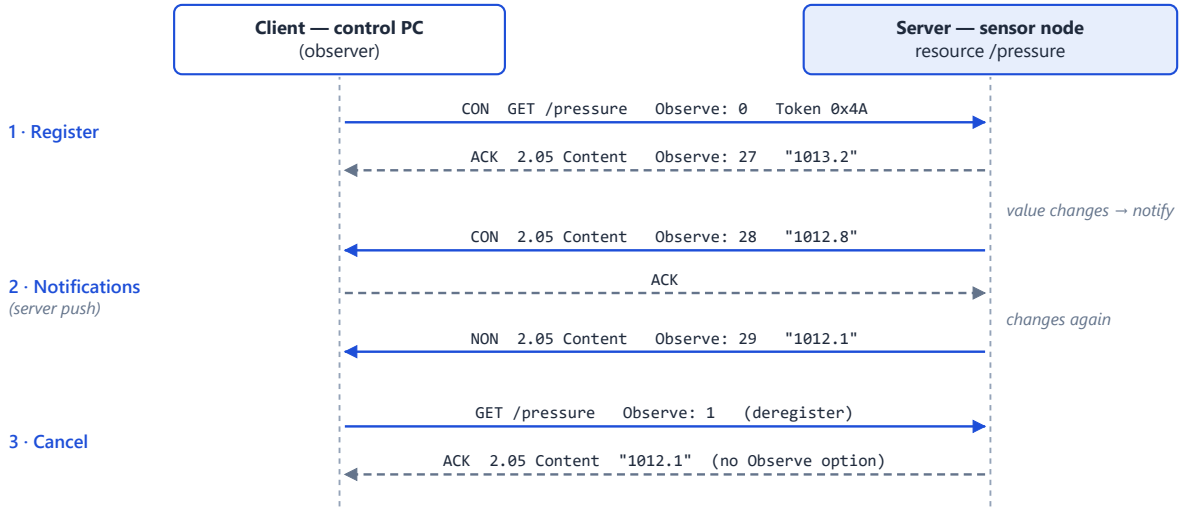


Figure D.4 *— CoAP Observe on /pressure: one registration, then the server pushes every change, like an MQTT subscription but without a broker.*

Example with the libcoap client: observe /pressure for 60 seconds

```
coap-client -m get -s 60 coap://192.168.1.50/pressure
```

Difference with MQTT: the observer talks **directly to the sensor**, so the sensor must stay reachable and keep its own list of observers; with MQTT, the broker does that work.

Exercise 2 Drones over MQTT, then REST (10 points)

The statement in short. The Internet of Flying Things (IoFT) brings UAVs (drones) into the IoT. A **Raspberry Pi** (model B), connected to the access point, manages the drones and acts as the **edge**: it talks to the microcontroller of each drone. The controller computer and the Pi communicate through an MQTT broker with three topics per drone:

- `/iod/uav01/commands` — commands sent to the drone;
- `/iod/uav01/processed` — confirmation that a command was executed;
- `/iod/uav01/state` — current state of the drone.

Commands are JSON strings, for example `{"COMMAND": "SET_MAX_ALTITUDE", "FEET": 20}`, `{"COMMAND": "TAKE_OFF"}`, `{"COMMAND": "ROTATE_RIGHT", "DEGREES": 90}` or `{"COMMAND": "LAND"}`. The drone already provides SET_MAX_ALTITUDE(...), TAKE_OFF(), MOVE_UP(), ROTATE_RIGHT(...), MOVE_RIGHT()..., and on the Pi a generic function `Faire_Commande(drone, commande, arguments)` applies a command to a drone. Below, the drone name (uav1) is used as is in the topic.

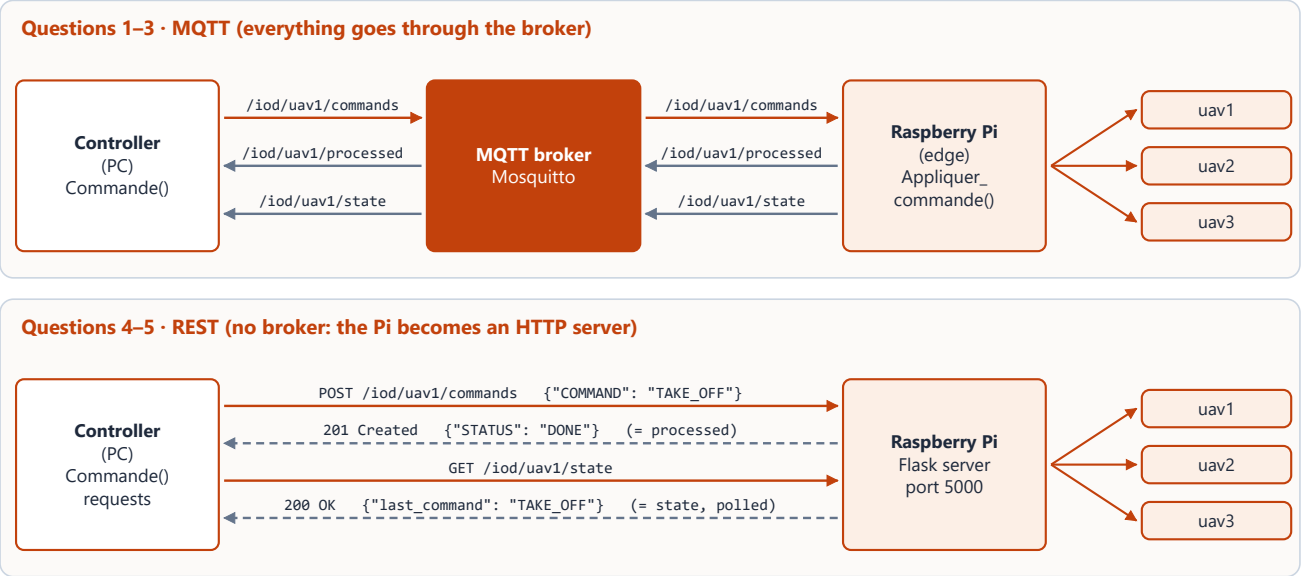


Figure D.5 *— The set-up of Exercise 2. Top: with MQTT, the controller and the Pi never talk directly. Bottom: with REST, the Pi becomes an HTTP server.*

Q1 *On the controller side, write a Python function Commande(drone, Chaine_Commande, Arguement_commande) that publishes the command and its parameters in the MQTT broker, in JSON as in the examples. Arguments: the drone name (e.g. uav1), the command as a string, and the command arguments (e.g. FEET for SET_MAX_ALTITUDE). Use the Eclipse Paho MQTT Python client.*

Answer. Build a dictionary with the key COMMAND plus the arguments, turn it into JSON with `json.dumps`, and publish it on `/iod/<drone>/commands` with QoS 1 so that the broker confirms reception.

Controller — commande_mqtt.py (paho-mqtt)

```
import json
import paho.mqtt.client as mqtt

BROKER = "192.168.1.10"                    # address of the Mosquitto broker

# paho-mqtt 2.x  (with paho-mqtt 1.x: mqtt.Client(client_id="controller"))
client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="controller")
client.connect(BROKER, 1883)
client.loop_start()                        # network loop in a background thread

def Commande(drone, Chaine_Commande, Arguement_commande=None):
    """Commande("uav1", "SET_MAX_ALTITUDE", {"FEET": 20})"""
    message = {"COMMAND": Chaine_Commande}
    if Arguement_commande:                 # e.g. {"FEET": 20} or {"DEGREES": 90}
        message.update(Arguement_commande)
    topic = "/iod/" + drone + "/commands"  # /iod/uav1/commands
    info = client.publish(topic, json.dumps(message), qos=1)
    info.wait_for_publish()                # returns when the broker sent its PUBACK
    return info.rc == mqtt.MQTT_ERR_SUCCESS

Commande("uav1", "SET_MAX_ALTITUDE", {"FEET": 20})
Commande("uav1", "TAKE_OFF")
Commande("uav1", "ROTATE_RIGHT", {"DEGREES": 90})
```

The network loop runs in a background thread (`loop_start()`), so `Commande` can be called many times on the same connection. With QoS 1, `wait_for_publish()` returns once the broker has answered with PUBACK.

Q2 *On the Raspberry Pi, a generic function Faire_Commande(drone, commande, Arguements) applies a command to a drone; for example Faire_Commande('uav0', 'Move_up', '') makes drone uav0 go up. Write a Python function Appliquer_commande() that receives the commands sent by the controller through the MQTT broker and applies them to the drones with Faire_Commande.*

Answer. The Pi subscribes once to `/iod/+/commands`: the + wildcard matches any drone name, so one subscription serves the whole fleet. In the message callback it reads the drone name from the topic, decodes the JSON, calls Faire_Commande, then publishes the confirmation on `/iod/<drone>/processed`.

Raspberry Pi — appliquer_mqtt.py (paho-mqtt)

```
import json
import paho.mqtt.client as mqtt

BROKER = "192.168.1.10"

def on_connect(client, userdata, flags, reason_code, properties=None):
    client.subscribe("/iod/+/commands", qos=1)       # + = any drone name

def on_message(client, userdata, msg):
    drone = msg.topic.split("/")[2]                  # "/iod/uav1/commands" -> "uav1"
    data = json.loads(msg.payload.decode())
    commande = data.pop("COMMAND")                   # what is left = the arguments
    try:
        Faire_Commande(drone, commande, data)        # e.g. ("uav1", "MOVE_UP", {})
        status = "DONE"
    except Exception as err:
        status = "ERROR: %s" % err
    reply = {"COMMAND": commande, "STATUS": status}
    client.publish("/iod/" + drone + "/processed", json.dumps(reply), qos=1)

def Appliquer_commande():
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2, client_id="raspberry-pi")
    client.on_connect = on_connect
    client.on_message = on_message
    client.connect(BROKER, 1883)
    client.loop_forever()                            # wait for commands, forever

Appliquer_commande()
```

- Subscribing inside `on_connect` subscribes again automatically after a reconnection.
- `msg.topic.split("/")` gives `['', 'iod', 'uav1', 'commands']`, so the drone name is at index 2.
- The `/state` topic can be fed the same way: a loop publishes each drone's state every second, with `retain=True` so that a new controller gets the last state at once.

Q3 *We want coordinated movements: the three drones uav1, uav2 and uav3 must make the same moves. Propose a single mosquitto_sub command line that launches, for example, MOVE_RIGHT for the 3 drones at the same time.*

Answer. Careful with the wording: `mosquitto_sub` only **receives**; sending (launching) a command is done with `mosquitto_pub`. A publish targets exactly one topic, and wildcards (+, #) are not allowed in a publish topic, so the three drones need a **shared topic**. Use the group topic `/iod/all/commands`: the Pi already receives it through its `/iod/+/commands` subscription.

Command line (Mosquitto clients)

```
# controller: ONE command line for the three drones
mosquitto_pub -h 192.168.1.10 -t "/iod/all/commands" -q 1 -m '{"COMMAND": "MOVE_RIGHT"}'

# controller: one subscription to follow the three confirmations
mosquitto_sub -h 192.168.1.10 -t "/iod/+/processed" -v
```

On the Pi, `on_message` treats the name `all` as the whole fleet and starts the three Faire_Commande calls in parallel threads, so the drones move together:

Raspberry Pi — change in on_message

```
import threading

FLEET = ["uav1", "uav2", "uav3"]

# in on_message, replace the single Faire_Commande call by:
targets = FLEET if drone == "all" else [drone]
threads = [threading.Thread(target=Faire_Commande, args=(d, commande, data))
           for d in targets]
for t in threads:
    t.start()                                        # the drones start together
for t in threads:
    t.join()
```

Q4 *We now switch from MQTT to REST. Write the Commande function of question 1 with the REST protocol.*

Answer. Without a broker, the controller sends an HTTP request straight to the Pi, which runs the REST server of question 5. Each MQTT topic becomes a resource:

MQTT (questions 1–3)	REST (questions 4–5)
Publish on /iod/uav1/commands	POST /iod/uav1/commands with the JSON command as body
Confirmation on /iod/uav1/processed	The HTTP response: 201 Created + {"STATUS": "DONE"}
State on /iod/uav1/state (pushed)	GET /iod/uav1/state (the controller polls)
Group topic /iod/all/commands	POST /iod/all/commands

Table D.2 *— From MQTT topics to REST resources.*

Controller — commande_rest.py (requests)

```
import requests

API = "http://192.168.1.20:5000"                    # Flask server on the Raspberry Pi

def Commande(drone, Chaine_Commande, Arguement_commande=None):
    """Commande("uav1", "SET_MAX_ALTITUDE", {"FEET": 20})"""
    message = {"COMMAND": Chaine_Commande}
    if Arguement_commande:
        message.update(Arguement_commande)
    r = requests.post(API + "/iod/" + drone + "/commands", json=message, timeout=5)
    r.raise_for_status()                            # exception on a 4xx / 5xx answer
    return r.json()                                 # the confirmation (old /processed)

print(Commande("uav1", "TAKE_OFF"))
# {'COMMAND': 'TAKE_OFF', 'STATUS': 'DONE', 'drone': 'uav1'}
```

Q5 *Write the same function with REST for the Raspberry Pi side, as in question 2 (the statement calls it Faire_Commande).*

Answer. On the Pi, the receiving side becomes a Flask server, built like the `add_country()` example of the annex: a POST route receives the JSON command, checks it, applies it with Faire_Commande and answers with the confirmation. Faire_Commande itself does not change; only the way the commands arrive changes.

Raspberry Pi — appliquer_rest.py (Flask)

```
from flask import Flask, request, jsonify

app = Flask(__name__)
FLEET = ["uav1", "uav2", "uav3"]
state = {d: {"last_command": None} for d in FLEET}

@app.post("/iod/<drone>/commands")
def Appliquer_commande(drone):
    if drone != "all" and drone not in FLEET:
        return {"error": "Unknown drone"}, 404
    if not request.is_json:
        return {"error": "Request must be JSON"}, 415
    data = request.get_json()
    commande = data.pop("COMMAND", None)
    if commande is None:
        return {"error": "COMMAND is missing"}, 400
    for d in (FLEET if drone == "all" else [drone]):
        Faire_Commande(d, commande, data)
        state[d]["last_command"] = commande
    return jsonify({"drone": drone, "COMMAND": commande, "STATUS": "DONE"}), 201

@app.get("/iod/<drone>/state")
def get_state(drone):
    if drone not in state:
        return {"error": "Unknown drone"}, 404
    return jsonify(state[drone])

app.run(host="0.0.0.0", port=5000)
```

Test: MOVE_RIGHT for the three drones with one request

```
curl -X POST http://192.168.1.20:5000/iod/all/commands \
     -H "Content-Type: application/json" -d '{"COMMAND": "MOVE_RIGHT"}'
```

KEY IDEA · MQTT OR REST FOR THE DRONES?

- **MQTT**: the controller and the Pi only know the broker; the Pi opens no port; the state is pushed; one message reaches many subscribers; QoS handles delivery.
- **REST**: simpler (no broker) and the confirmation comes back in the same HTTP response, but the controller must know the Pi's address, the state must be polled, and a group command needs its own route (/iod/all).